

多变换域联合的信号调制特征识别与滤波优化系统

数字信号处理综合大作业实践报告（题目三：创新拓展型）

李宸威 学号：202464820504

班级：24 级超级机器人珠峰班 指导教师：俞祝良

2026 年 8 月 24 日

摘要

本报告围绕数字调制通信信号的处理需求，设计并实现了一个多变换域联合的信号调制特征识别与滤波优化系统。系统以 Python 为实现平台，完整覆盖离散信号建模、时域卷积、DTFT/DFT/Z 变换多域联合分析、FIR/IIR 滤波器设计与迭代优化、NLMS 自适应滤波、调制方式自动识别与解调级误码率评估等环节。针对 ASK/FSK/PSK 三种数字调制信号，系统在 10 dB 信噪比下实现了 100% 的自动识别准确率；滤波器参数三维网格扫描（阶数 $\times$ 窗函数 $\times$ 截止频率）使信噪比改善最高达 9.60 dB；消融实验定量证明多域联合识别相比最佳单一域取得 +31.1% 的准确率增益。报告详细阐述各环节的理论依据、实现方法与实验结果，关键程序代码见附录。

目录

1	引言	3
1.1	选题背景	3
1.2	总体设计	3
1.3	主要工作与工作量	3
2	基本理论	4
2.1	数字调制信号模型	4
2.2	时域分析与卷积平滑	5
2.3	DTFT 与 DFT 及其关系	5
2.4	Z 变换与零极点分析	6

目录	2
2.5 FIR/IIR 滤波器设计	6
2.6 LMS 与 NLMS 自适应滤波	6
2.7 解调与误码率 (BER)	7
2.8 调制识别特征体系	7
3 系统实现	8
3.1 开发环境与工程结构	8
3.2 各模块关键实现细节	8
3.3 关键问题与排查过程	9
4 实验结果与分析	10
4.1 时域分析与卷积平滑	10
4.2 多变换域联合分析	10
4.3 滤波器设计与迭代优化	11
4.4 NLMS 自适应滤波	16
4.5 调制方式自动识别	17
4.6 综合性能评估: SNR、保真度、BER 与运算复杂度	18
4.7 SNR 扫描蒙特卡洛实验	20
4.8 单一域 vs 多域联合消融实验	20
5 创新拓展与总结	22
5.1 创新点	22
5.2 各变换工具的分工与协同	22
5.3 局限与展望	23
A 附录: 关键程序代码	23
A.1 全局参数配置 (config.py, 节选)	23
A.2 信号生成 (utils/generator.py)	24
A.3 多域分析核心算法 (utils/analyzer.py, 节选)	28
A.4 滤波器设计与自适应滤波 (utils/filter_design.py, 节选)	32
A.5 解调与误码率 (utils/demodulator.py)	38
A.6 特征提取与调制识别 (utils/recognizer.py, 节选)	41
A.7 主流程 (main.py)	48

1 引言

1.1 选题背景

在现代数字通信系统中，信号经信道传输后不可避免地叠加噪声，接收端需要完成两个核心任务：一是通过滤波尽可能恢复纯净信号，二是自动识别信号的调制方式以便正确解调。单一域（仅时域或仅频域）的分析手段往往只能刻画信号的局部特征，难以应对调制方式间的相似性（如 ASK 与 PSK 的频谱均为单峰结构）。本题要求建立“时域-频域-Z 域”三维联合分析思维，完成调制信号的特征分析、噪声抑制与自动识别，并对滤波器性能进行迭代优化。

1.2 总体设计

系统采用模块化流水线结构，共 11 个处理步骤，全部由 Python（NumPy、SciPy、Matplotlib）实现。整体流程与模块对应关系如表 1 所示。

表 1：系统处理流水线与模块对应关系

步骤	内容	核心模块/函数	输出
1	调制信号建模与生成	generator	纯净/含噪信号
2	时域分析与卷积平滑	analyzer	图 01-02
3	DTFT/DFT/Z 域多域分析	analyzer	图 03-05、13、15
4	FIR/IIR 滤波器设计	filter_design	图 06-07、系统函数
5	滤波器参数迭代优化	filter_design.iterative_optimize	图 09、09b
6	NLMS 自适应滤波	filter_design.lms_filter	图 10、10b
7	调制方式自动识别	recognizer.classify_signal	图 11
8	性能评估（SNR/BER/复杂度）	recognizer	图 12
9	SNR 扫描蒙特卡洛实验	recognizer.run_snr_sweep	图 14
10	单域 vs 多域消融实验	recognizer.run_domain_ablation	图 16
11	评估报告生成	recognizer.generate_report	logs.txt

实验参数统一在 config.py 中管理：采样率 $F_s = 10\text{ kHz}$ ，符号速率 10 Baud（每符号 1000 点），信息长度 20 bit，默认信噪比 $\text{SNR} = 10\text{ dB}$ ，随机种子 42 保证结果可复现。三种调制的载波参数为：ASK 载频 $f_c = 1\text{ kHz}$ （OOK, $A_1=1, A_0=0$ ）；FSK 两个符号频率 $f_1 = 800\text{ Hz}$ 、 $f_2 = 1200\text{ Hz}$ ；PSK（BPSK）载频 1 kHz、相位 $\{0, \pi\}$ 。由于符号频率均为符号率整数倍，逐符号生成的 FSK 相位天然连续（等效 CPFSK），频谱双峰结构干净。

1.3 主要工作与工作量

本作业围绕题目五项实施要点完成的主要工作如下：

1. **系统搭建**: 自主设计并实现覆盖“建模–分析–滤波–识别–评估”全链路的 DSP 仿真系统, 含 7 个源模块、约 3660 行 Python 代码, 单命令复现全部实验, 自动产出 29 张结果图与 1 份六章节量化评估报告;
2. **信号建模**: 实现 ASK/FSK/PSK 三种数字调制信号模型与 AWGN 信道 (式 (1)–(4)), 比特序列与噪声流分离管理, 支持可控复现;
3. **多域分析**: 实现时域卷积平滑与统计特征、DFT/DTFT 频谱分析、基于 AR 参数模型的 Z 域零极点估计, 并完成三种变换应用差异的对比分析 (图 6);
4. **滤波优化**: 设计 FIR/IIR 带通滤波器并完成“阶数 $\times$ 窗函数 $\times$ 截止频率”三维网格 (84 种组合/调制) 迭代优化, 以保真度与归一化 ΔSNR 加权评分自动选型;
5. **自适应滤波**: 实现标准 LMS 与 NLMS 两种算法, 完成步长扫描、收敛性分析、权值演化追踪与按条件择优策略;
6. **解调评估**: 构建三种调制的完整解调链路 (包络检波/相关/相干解调), 建立解调级误码率 (BER) 评估体系, 并量化各方法的运算复杂度;
7. **系统实验**: 完成 7 个 SNR 点 $\times$ 5 次蒙特卡洛的性能扫描实验, 以及“仅时域/仅频域/仅 Z 域/多域联合”四模式消融实验, 定量回答“多域联合优势”问题;
8. **问题排查**: 针对开发中发现的 4 个典型问题完成根因分析与修复验证 (详见 §3.3)。

本系统完整源代码、实验结果图表与评估报告已开源于 GitHub 仓库: <https://github.com/lcw578/dsp-course-project>。

2 基本理论

2.1 数字调制信号模型

设二进制比特序列 $b_m \in \{0, 1\}$, $m = 0, 1, \dots, M-1$, 每符号采样点数 $N_s = F_s T_s = 1000$ 。

(1) **ASK (幅移键控, OOK)**: 用幅度携带信息,

$$s_{\text{ASK}}(n) = A(b_m) \cos(2\pi f_c n T_s), \quad A(b_m) = \begin{cases} A_1 = 1, & b_m = 1 \\ A_0 = 0, & b_m = 0 \end{cases} \quad (1)$$

(2) **FSK (频移键控)**: 用频率携带信息,

$$s_{\text{FSK}}(n) = \cos(2\pi f_{b_m} n T_s), \quad f_{b_m} = \begin{cases} f_1 = 800 \text{ Hz}, & b_m = 0 \\ f_2 = 1200 \text{ Hz}, & b_m = 1 \end{cases} \quad (2)$$

(3) PSK (相移键控, BPSK): 用相位携带信息,

$$s_{\text{PSK}}(n) = \cos(2\pi f_c n T_s + \varphi_{b_m}), \quad \varphi_{b_m} = \begin{cases} 0, & b_m = 0 \\ \pi, & b_m = 1 \end{cases} \quad (3)$$

信道噪声模型 (AWGN): 叠加加性高斯白噪声

$$x(n) = s(n) + v(n), \quad v(n) \sim \mathcal{N}(0, \sigma_v^2), \quad \sigma_v^2 = \frac{P_s}{10^{\text{SNR}/10}} \quad (4)$$

其中 $P_s = \frac{1}{N} \sum_n s^2(n)$ 为信号功率。SNR = 10 dB 时 $\sigma_v^2 = P_s/10$ 。

2.2 时域分析与卷积平滑

卷积是 LTI 系统分析的基础运算。时域平滑采用矩形核均值滤波器:

$$h[k] = \frac{1}{L}, \quad k = 0, 1, \dots, L-1, \quad y[n] = \sum_k x[k] h[n-k] = \frac{1}{L} \sum_{k=n-L+1}^n x[k] \quad (5)$$

窗长 L 越大平滑 (去噪) 能力越强, 但会模糊信号边缘细节, 实验中对 $L \in \{5, 11, 21, 51\}$ 进行对比。

信号包络由 Hilbert 变换构造解析信号获得:

$$\tilde{x}(n) = x(n) + j\hat{x}(n), \quad \text{env}(n) = |\tilde{x}(n)| \quad (6)$$

其中 $\hat{x}$ 为 Hilbert 变换。包络的变异系数 $\text{CV} = \sigma_{\text{env}}/\mu_{\text{env}}$ 是区分幅度键控与恒包络调制的关键时域特征。

2.3 DTFT 与 DFT 及其关系

DTFT (离散时间傅里叶变换) 是连续谱:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}, \quad \omega \in [-\pi, \pi) \quad (7)$$

DFT (离散傅里叶变换) 是有限长序列的离散谱:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (8)$$

二者的核心关系为: DFT 是 DTFT 在 $\omega_k = 2\pi k/N$ 处的等间隔采样, 频率分辨率 $\Delta f = F_s/N$ 。本系统 $N = 20000$, $\Delta f = 0.5$ Hz。工程实现中, DTFT 通过零填充 FFT 在密集频点上逼近; DFT 直接由 FFT 计算。DFT 谱线支撑峰值检测 (调制识别的频域核心特征), DTFT 连续谱形则直观呈现带宽与旁瓣结构。

2.4 Z 变换与零极点分析

序列的 Z 变换定义为 $X(z) = \sum_n x[n]z^{-n}$ 。对 LTI 系统, 传递函数 $H(z) = B(z)/A(z)$ 的零极点分布完全刻画其频率选择性: 极点在单位圆内靠近单位圆处对应谐振峰, 极点角位置 θ 映射模拟频率 $f \approx \theta F_s/(2\pi)$ 。

离散信号本身不具有有理形式的 Z 变换, 本系统采用 **AR 参数模型** 对信号进行 Z 域建模:

$$x(n) = \sum_{k=1}^p a_k x(n-k) + e(n) \iff H_{\text{AR}}(z) = \frac{1}{1 - \sum_{k=1}^p a_k z^{-k}} \quad (9)$$

AR 系数 a_k 由 Yule-Walker 方程 $R\mathbf{a} = \mathbf{r}$ 求解 (自相关矩阵 R 为 Toeplitz 结构, 用 Levinson-Durbin 递推高效求解), 随后对 $A(z)$ 求根得到极点。主极点模长 $r = |p_{\max}|$ 反映谱线锐度 (品质因数): 恒包络调制 (FSK/PSK) 的载波谱线极尖锐, $r \rightarrow 1$; 而 ASK 因通断键控使谱线展宽, r 明显下降——这是 Z 域区分幅度键控与恒包络调制的物理依据。

2.5 FIR/IIR 滤波器设计

FIR 窗函数法: 以理想带通响应 $h_d[n]$ 与窗函数 $w[n]$ 相乘截断:

$$h_d[n] = \frac{\sin(\omega_2(n-\alpha)) - \sin(\omega_1(n-\alpha))}{\pi(n-\alpha)}, \quad h[n] = h_d[n]w[n] \quad (10)$$

其中 $\alpha = N/2$ 保证线性相位。本系统对比 Hamming、Hann、Blackman、Kaiser 四种窗。

IIR 滤波器: 基于模拟原型经双线性变换 $s \leftarrow \frac{2}{T} \frac{1-z^{-1}}{1+z^{-1}}$ 设计。Butterworth 幅度平方函数为

$$|H(j\Omega)|^2 = \frac{1}{1 + (\Omega/\Omega_c)^{2N}} \quad (11)$$

另对比 Chebyshev I 型 (通带等纹波) 与椭圆滤波器 (双带等纹波、过渡带最窄)。

零相位滤波: 使用 `filtfilt` (前向 + 后向滤波), 等效频率响应为 $|H_{\text{eff}}(e^{j\omega})| = |H(e^{j\omega})|^2$, 相位严格为零, 完全消除群延迟引起的波形失真, 代价是乘加次数加倍。

2.6 LMS 与 NLMS 自适应滤波

自适应噪声对消 (ANC) 结构: 参考信道输入与噪声相关的信号 $x(n)$, 期望响应 $d(n) = s(n) + v(n)$, 自适应滤波器输出噪声估计 $\hat{v}(n) = \sum_k w_k x(n-k)$, 误差

$$e(n) = d(n) - \hat{v}(n) = s(n) + (v(n) - \hat{v}(n)) \quad (12)$$

当权向量收敛到维纳解时 $v(n) - \hat{v}(n) \rightarrow 0$, 误差即为去噪信号。

LMS 算法按最速下降方向更新权值：

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n) \mathbf{x}(n) \quad (13)$$

收敛条件为 $0 < \mu < 1/(MP_x)$ (M 为滤波器阶数, P_x 为输入功率)。标准 LMS 步长与输入功率强耦合：白噪声输入下权值需收敛到类冲激响应，收敛时间常数约为 $M/(2\mu)$ 量级，短记录内常来不及收敛。

NLMS (归一化 LMS) 对更新步长按输入向量能量归一化：

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\mu e(n) \mathbf{x}(n)}{\mathbf{x}^T(n) \mathbf{x}(n) + \delta}, \quad \mu \in (0, 2) \quad (14)$$

步长与输入功率无关，收敛速度大幅提升且对输入电平变化鲁棒。代价是稳态失调随 μ 增大而增大，存在“收敛速度-稳态精度”折中，实验通过步长扫描定量刻画。

2.7 解调与误码率 (BER)

波形级指标 (SNR、相关系数) 之外，系统进一步实现解调级评估：

- **ASK 包络检波**：Hilbert 包络按符号积累能量 E_m ，与自适应门限比较判决；
- **FSK 相关解调**：每符号分别与两个本地载波作相关运算 $|\sum_n s(n) \cos(2\pi f_i n T_s)|$ ，择大判决；
- **PSK 相干解调**：每符号与本地同相载波相关，按极性判决。

误码率定义为 $\text{BER} = N_{\text{err}}/N_{\text{bits}}$ ，直接反映滤波对通信正确性的改善。

2.8 调制识别特征体系

多域联合识别使用三类特征 (表 2)。

表 2: 调制识别特征体系

特征域	特征	定义	判别意义
时域	包络变异系数 CV	$\sigma_{\text{env}}/\mu_{\text{env}}$	ASK 大 (> 0.5)，恒包络小 (< 0.3)
时域	相位跳变率	$P(\Delta\phi > \pi/2)$	PSK 符号边界相位反转
时域	瞬时频率标准差	σ_{f_i}	FSK 双频切换使 σ_{f_i} 大
频域	显著谱峰数量	峰值检测 (阈值 0.3)	FSK 双峰，ASK/PSK 单峰
频域	谱集中度	主峰 ± 100 Hz 能量占比	单载波调制能量集中
Z 域	主极点半径 r	AR 模型 $ p_{\text{max}} $	恒包络 $r \geq 0.96$ ，ASK 谱线展宽

分类器采用加权投票规则：每条规则命中时为对应类别累加分数，最终取得分最高者为识别结果，置信度为该分类得分占总得分比例。

3 系统实现

3.1 开发环境与工程结构

开发与实验环境如表 3 所示，工程按“参数-生成-分析-滤波-解调-识别”职责划分为 7 个模块（合计约 3660 行），主流程入口为 `main.py`，单命令即可复现全部实验与图表。

表 3: 开发环境

项目	配置
操作系统	Windows x64
编程语言	Python 3.11.15
数值计算	NumPy 2.4.6、SciPy 1.17.1
可视化	Matplotlib 3.9.2（Agg 后端，SimHei 中文字体）
报告排版	L ^A T _E X（XeLaTeX + ctex，MiKTeX 25.12）
版本管理	Git（GitHub 仓库 https://github.com/lcw578/dsp-course-project ）

表 4: 工程模块划分

模块	行数	职责
<code>config.py</code>	112	全局参数中心（采样/载波/滤波器/自适应/SNR 扫描）
<code>utils/generator.py</code>	260	比特序列、ASK/FSK/PSK 调制、AWGN 信道
<code>utils/analyzer.py</code>	701	卷积平滑、DFT/DTFT/Z 域分析、多域可视化
<code>utils/filter_design.py</code>	857	FIR/IIR 设计、迭代优化、LMS/NLMS 自适应滤波
<code>utils/demodulator.py</code>	175	三种调制解调与 BER 计算
<code>utils/recognizer.py</code>	1036	特征提取、分类识别、性能评估、扫描/消融实验
<code>main.py</code>	516	11 步主流程调度

3.2 各模块关键实现细节

- （1）信号生成与信道建模。三种调制波形按符号分段生成；由于各载频均为符号率（10 Hz）的整数倍，符号边界处载波相位差为零，逐符号独立生成即等效连续相位调制。比特序列由独立种子 `RandomState(42)` 驱动，而 AWGN 使用全局随机流——二者分离使不同试验间比特保持一致、噪声实现相互独立，便于蒙特卡洛统计。
- （2）多域分析。DFT 单边谱归一化时对 DC 与 Nyquist 分量不作加倍处理，避免端点幅度失真；DTFT 通过 4096/8192 点零填充 FFT 在密集频点逼近；AR 建模取信号中段 4000 点（覆盖 4 个符号），保证 FSK 的两个载频成分都进入自相关估计，模型

阶数取 6——阶数过高（如 12）会对带通噪声过拟合，产生大量虚假谐振极点，反而破坏特征区分度（排查过程见 §3.3 问题 3）。

(3) 滤波器设计与优化。 `firwin(numtaps=order+1)` 使参数“阶数”与教科书定义严格一致；`filtfilt` 零相位滤波对短信号保护 `padlen` 并自动回退 `lfilter`。迭代优化的综合评分设计为

$$\text{Score} = 0.6 \times \text{corr} + 0.4 \times \frac{\min(\max(\Delta\text{SNR}, 0), 15)}{15} \quad (15)$$

将 SNR 改善以 15 dB 为参考上限归一化到 $[0, 1]$ ，与保真度等权融合，避免原始评分公式中 $\text{SNR}/50$ 归一化基准无依据的问题。

(4) NLMS 自适应滤波。 ANC 参考信道取含当前时刻样本的噪声序列，此时互相关向量 $p[0] = \sigma_v^2 > 0$ ，维纳解为类冲激响应 $\mathbf{w}^* = [1, 0, \dots, 0]^*$ （该设计的关键性见 §3.3 问题 1）；迭代未启动的前 M 个样本对期望信号透传输出。SNR 扫描实验中，NLMS 在候选步长 $\{0.01, 0.05, 0.1, 0.5\}$ 内按当前噪声实现下的输出 SNR 择优——深噪区选大步长保证快速收敛，高 SNR 区选小步长压低稳态失调，体现“收敛速度–稳态精度”折中的工程化处理。

(5) 解调与 BER 评估。 ASK 采用每符号能量的 10%/90% 分位数中点作为判决门限，对离群帧（如自适应滤波收敛瞬态造成的能量异常）鲁棒（§3.3 问题 2）；FSK/PSK 相关解调利用载波整周期特性直接作内积判决，无需载波恢复环路；BER 统计前按符号数对齐收发序列。

(6) 调制识别分类器。 规则权重按证据强度分级：强证据（如 FSK 频谱双峰）计 3 分，中等证据（包络稳定、相位跳变）计 1.5–2.5 分，弱证据计 0.5–1 分；分类器通过 `domain` 参数支持仅时域/仅频域/仅 Z 域/多域联合四种模式，为消融实验提供统一接口，保证对比公平。

3.3 关键问题与排查过程

题目强调“问题排查能力”的培养。开发与调试过程中定位并修复了以下 4 个典型问题，每个问题均遵循“现象 → 根因分析 → 修复 → 验证”的完整流程。

问题 1：标准 LMS 自适应滤波完全失效。 现象： $\Delta\text{SNR} \approx -0.1 \text{ dB}$ ，稳态 MSE (0.27) 远大于噪声功率 (0.025)，滤波器输出趋近于零。根因：原实现中参考向量仅取时延 $1 \sim M$ 的样本 $\mathbf{x}(n) = [x(n-1), \dots, x(n-M)]$ ，而白噪声各时刻互不相关，互相关向量 $\mathbf{p} = \mathbb{E}[d(n)\mathbf{x}(n)] = \mathbf{0}$ ，维纳解 $\mathbf{w}^* = R^{-1}\mathbf{p} = \mathbf{0}$ ——自适应滤波器“学到”的最优解恰是零输出，结构上不可能去噪。修复：参考向量纳入当前时刻样本（时延 $0 \sim M-1$ ），使 $p[0] = \sigma_v^2 > 0$ ，维纳解变为类冲激 $\mathbf{w}^* = [1, 0, \dots, 0]$ 。验证： ΔSNR 由 -0.1 dB 提升至 $+9.4 \text{ dB}$ ，权值演化曲线确认 $w[0] \rightarrow 1$ 、其余 $\rightarrow 0$ ，与理论维纳解一致。

问题 2：NLMS 在深噪区 BER 异常偏高。 现象： -25 dB 时 NLMS 的 ΔSNR 高达 $+25 \text{ dB}$ ，但 ASK 解调 BER 反而高达 0.4。根因：自适应收敛瞬态（前约 200 样本）使首个符号帧的能量膨胀至正常帧的约 3 倍；而原 ASK 判决门限取（最大 + 最小）帧

能量中点，被该离群帧拉高至超过全部正常 bit-1 帧能量，造成系统性误判为 0。修复：判决门限改为能量 10%/90% 分位数中点（对离群帧鲁棒），并将迭代未启动的前 M 个样本改为透传而非置零。验证：-25 dB 时 ASK 的 BER 降至 0.0067 量级，三种调制 BER 均单调合理。

问题 3: Z 域特征无判别力。现象：仅 Z 域识别模式的准确率恒为随机水平（33%），连特征最显著的 FSK 也无法识别。根因：其一，AR 建模窗口仅取信号前 1000 点（恰为 1 个符号），单符号窗口内不存在 FSK 的第二个载频成分，双极点对特征无从产生；其二，模型阶数取 12 时对带通信号过拟合，三种调制均产生 6 个“强极点对”，特征彻底失去区分度。修复：窗口改为信号中段 4000 点（覆盖 4 个符号），模型阶数降至 6，并以主极点半径 $r = |p_{\max}|$ 作为 Z 域特征。验证：ASK 的 $r \approx 0.93 \sim 0.96$ 与恒包络调制的 $r \approx 0.998$ 形成稳定分界，Z 域证据成功参与融合判决。

问题 4: 深噪区 FIR 的 BER 反而高于未滤波。现象：-25 dB 时 FIR 平均 BER (0.233) 高于未滤波 (0.200)，与“滤波有益”的直觉矛盾。根因分析：该区间内 ASK 能量检测在滤波前后均被噪声完全淹没（各自贡献约 0.5 的错误率），FSK/PSK 相关解调经 FIR 后接近无误码；表观“恶化”主要来自每单元仅 $3 \times 3 \times 20 = 180 \text{ bit}$ 的蒙特卡洛统计涨落（ $\pm 0.03 \sim 0.05$ ）。处理：将每 SNR 点试验次数由 3 增至 5 以收窄方差，并在报告中如实呈现深噪区结论的统计不确定性——这本身即是蒙特卡洛方法教学意义的体现。

4 实验结果与分析

4.1 时域分析与卷积平滑

图 1 给出三种调制纯净与含噪波形（前 5 个符号）。ASK 的 OOK 结构使包络在 0 与 1 间跳变；FSK/PSK 为恒包络，二者波形幅度相同、仅频率/相位不同——这解释了时域特征 CV 的判别力。图 2 展示不同窗长卷积平滑效果： $L = 5$ 保留细节但去噪弱， $L = 51$ 平滑强但模糊符号边缘，验证了卷积平滑的折中特性。

时域统计特征定量印证了上述观察：ASK 的包络方差 (0.227) 比 FSK/PSK (≤ 0.0001) 高出三个数量级；ASK 的过零数 (1410) 显著低于 FSK/PSK (3760/4010)，原因是 OOK 符号内幅度为零的区段不产生过零。这些统计量的显著差异正是第 2.7 节识别特征的直接来源。

4.2 多变换域联合分析

图 3-5 分别为 DFT 幅度/相位谱、DTFT 连续谱与 Z 域零极点分布。DFT 频域中 FSK 呈现间隔 400 Hz 的双峰，ASK/PSK 为单峰；Z 域中三种信号的强极点均落在各自载频对应角度附近。图 6 将 DFT 谱线叠加在 DTFT 连续谱上，直观展示“DFT 是 DTFT 的等间隔采样”（谱线间隔 $F_s/N = 0.5 \text{ Hz}$ ）；图 7 为时域-频域-Z 域三列联合对

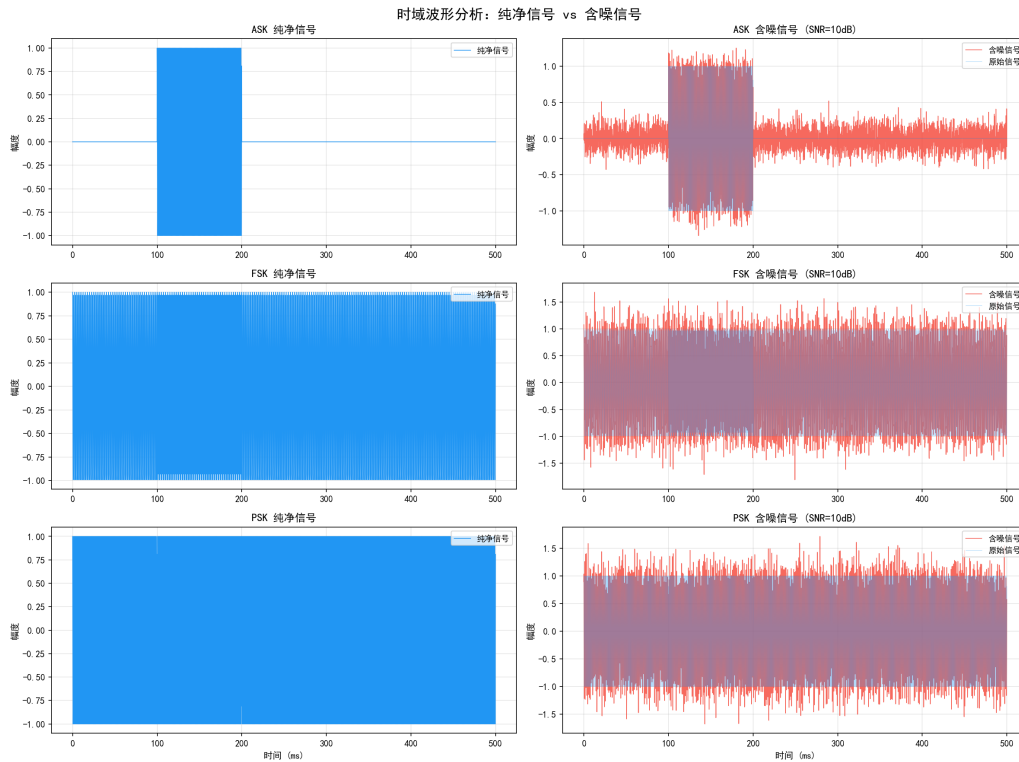


图 1: 三种调制信号时域波形 (纯净 vs 含噪, SNR=10 dB)

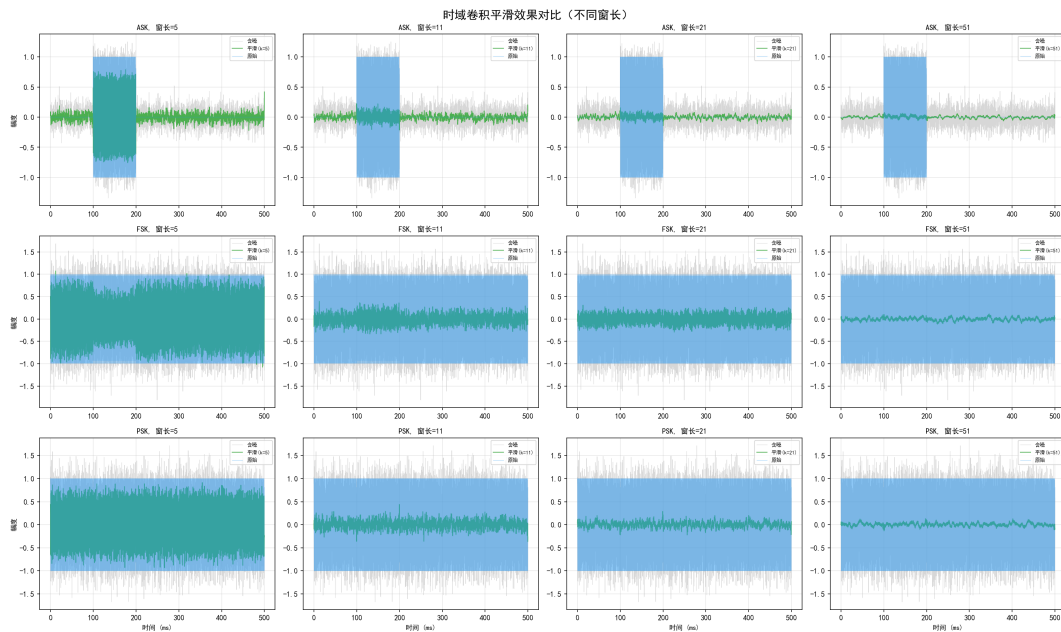


图 2: 不同窗长的卷积平滑效果对比

比总览。

4.3 滤波器设计与迭代优化

以 FSK 带通 (500–1500 Hz) 为例, FIR (64 阶 Hamming) 与 IIR (4 阶 Butterworth) 的幅频/相频特性如图 8。FIR 严格线性相位; IIR 阶数低、过渡带窄但相位非线性——

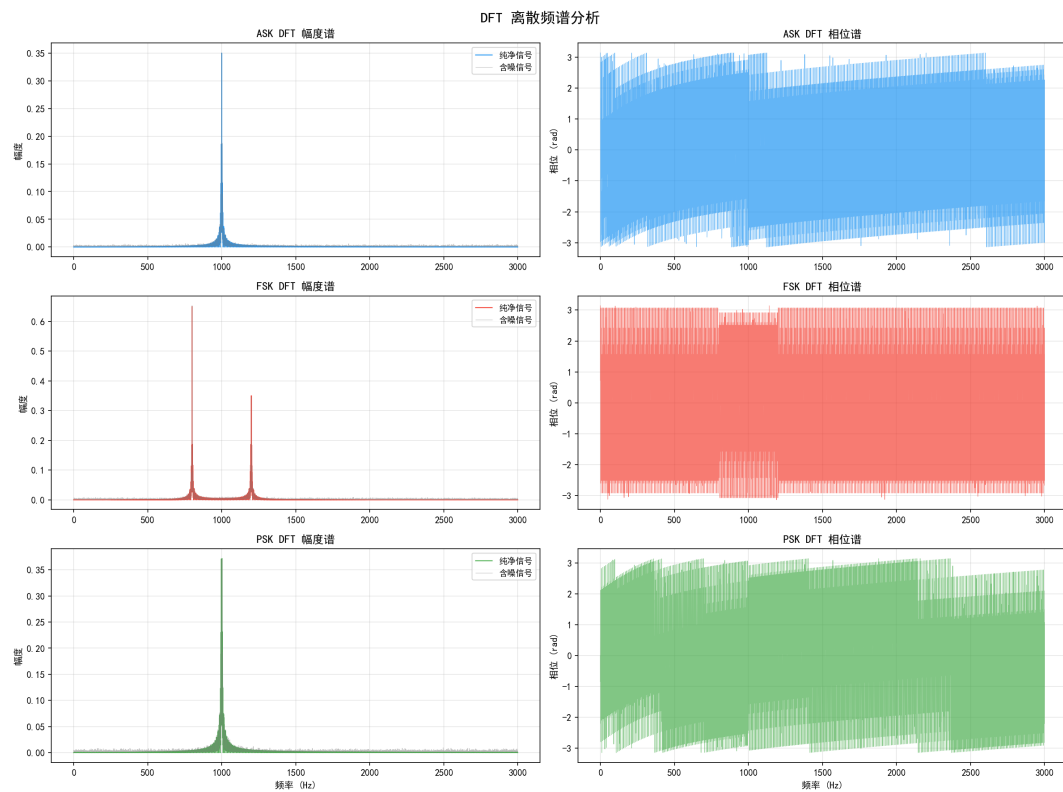


图 3: DFT 离散频谱 (左: 幅度谱, 右: 相位谱)

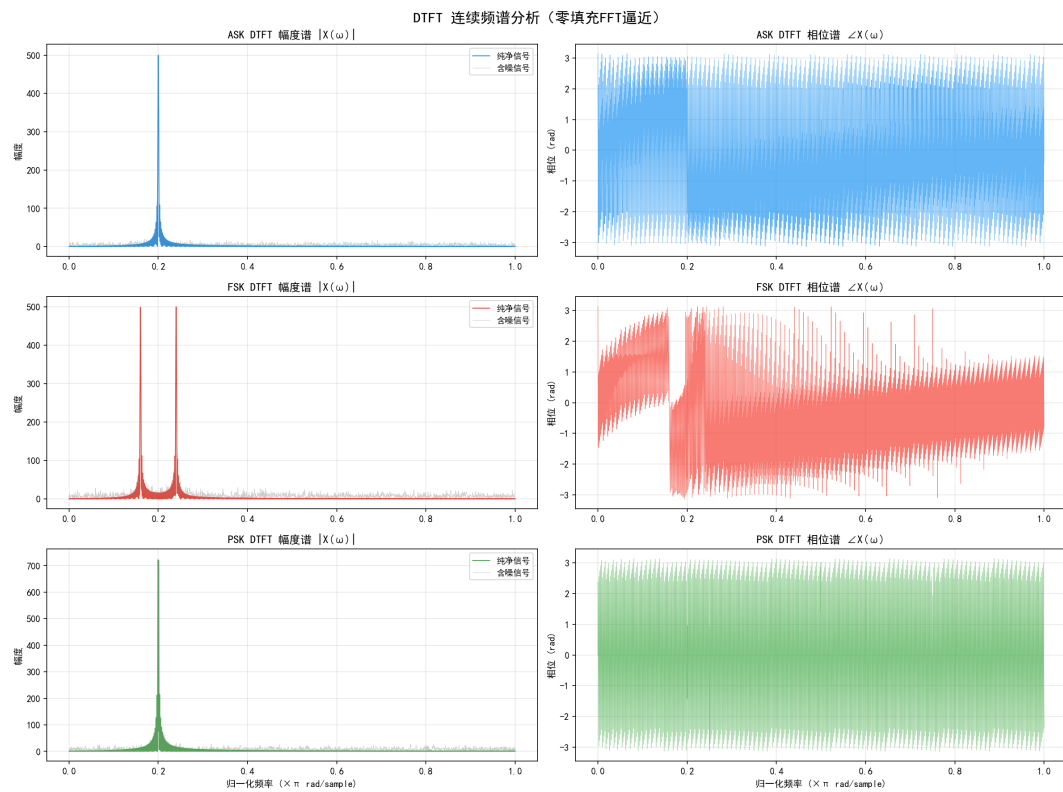


图 4: DTFT 连续频谱 (零填充 FFT 逼近)

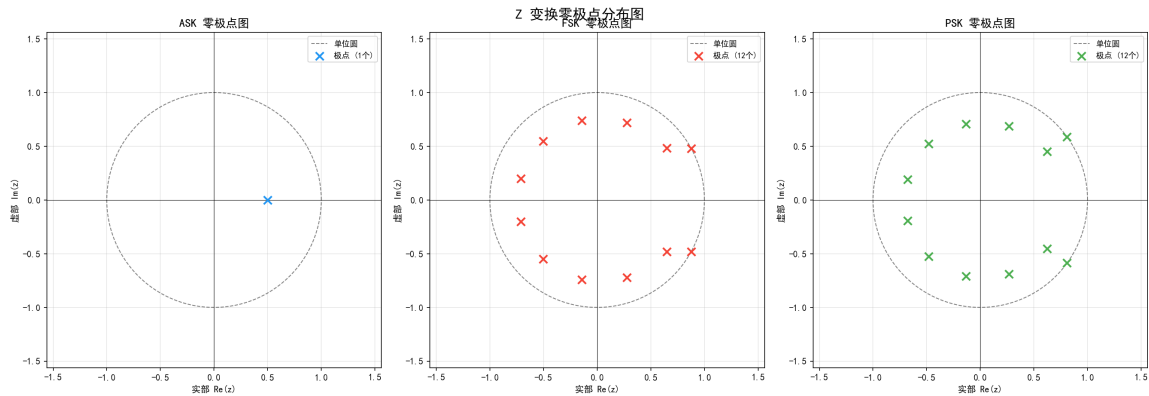


图 5: Z 变换零极点分布 (AR 模型建模)

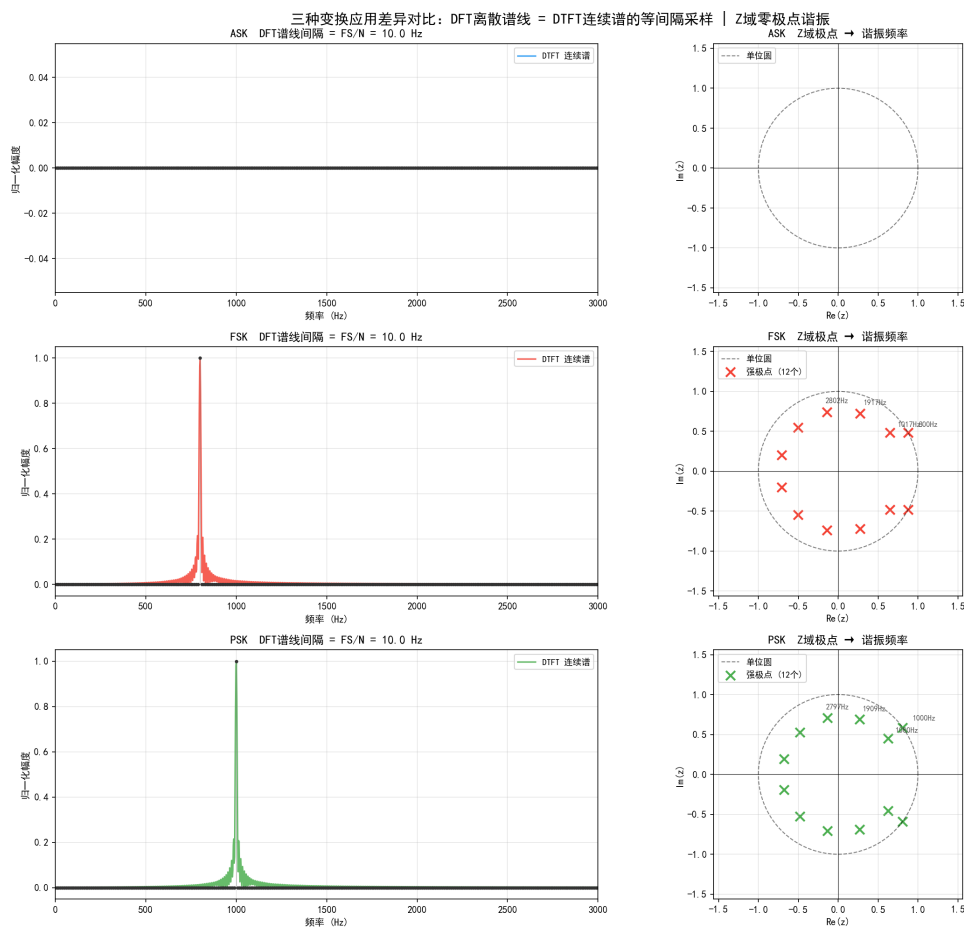


图 6: 三种变换应用差异对比: DFT 谱线与 DTFT 连续谱的采样关系 (左), Z 域极点谐振频率 (右)

系统采用 `filtfilt` 零相位滤波消除其影响。IIR 系统函数推导输出示例 ($N(z)/D(z)$ 多项式形式) 由程序自动打印。

三维参数扫描结果 (图 9) 汇总于表 5: ASK/PSK 最优解均为 **FIR 32 阶 Kaiser**, FSK 最优为 **FIR 64 阶 Blackman**——FSK 需要分辨间隔 400 Hz 的双峰, 更高阶数带来的更窄过渡带收益大于计算代价; ASK/PSK 为单峰信号, 32 阶 Kaiser (可调 β 参数) 在保真度与阻带衰减间取得最佳平衡。三种调制的最优带宽缩放均为 $\times 1.0$, 表明名

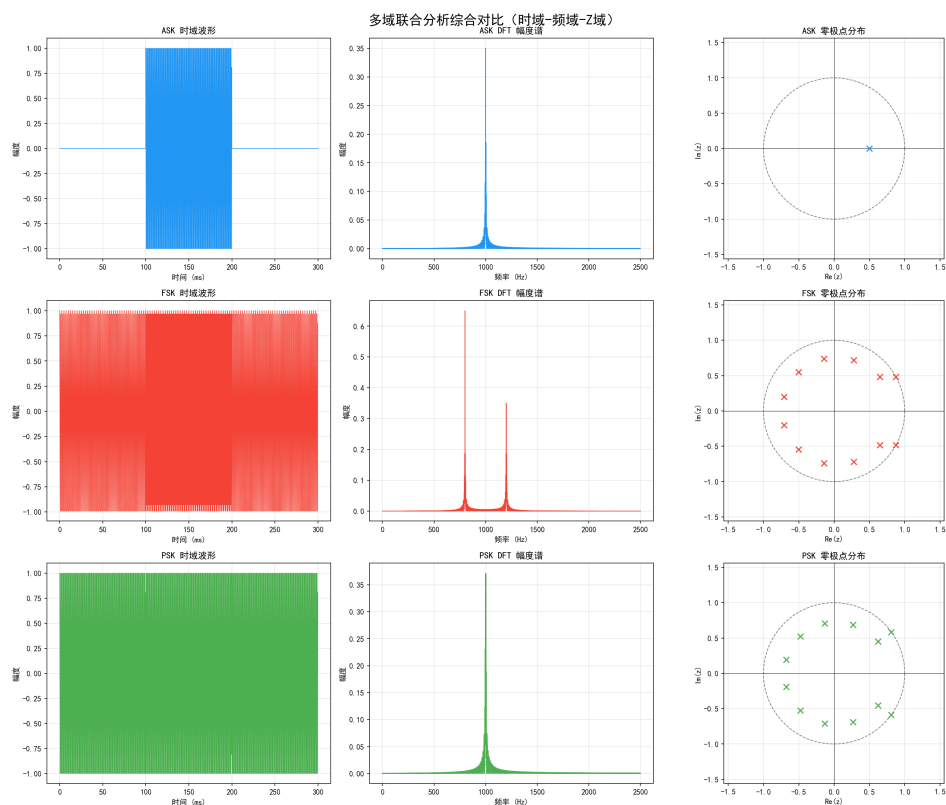


图 7: 时域-频域-Z 域多域联合对比总览

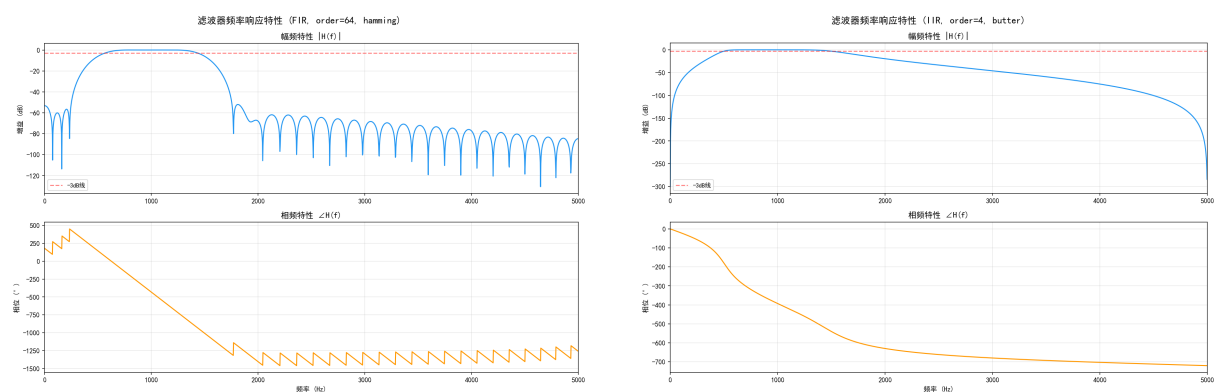


图 8: FIR (左, 64 阶 Hamming) 与 IIR (右, 4 阶 Butterworth) 带通滤波器幅频/相频特性

义带宽设计已接近最优, $\pm 20\%$ 偏离引起 $0.1 \sim 0.3 \text{ dB}$ 的次级差异。IIR 全部组合均未胜出: 其非线性相位虽被 `filtfilt` 消除, 但同等阶数下阻带抑制弱于高阶 FIR。最优滤波器的幅频特性与 Z 域零极点形态见图 10 (以 FSK 为例), 零点均匀分布于阻带对应角度, 极点位于通带边缘附近, 符合带通选择性的零极点设计原理。

表 5: 迭代优化最优滤波器组合（84 种组合/调制）

调制	最优组合	带宽缩放	ΔSNR (dB)	保真度
ASK	FIR 32 阶 Kaiser	$\times 1.0$	+9.60	0.9945
FSK	FIR 64 阶 Blackman	$\times 1.0$	+8.10	0.9924
PSK	FIR 32 阶 Kaiser	$\times 1.0$	+9.27	0.9940

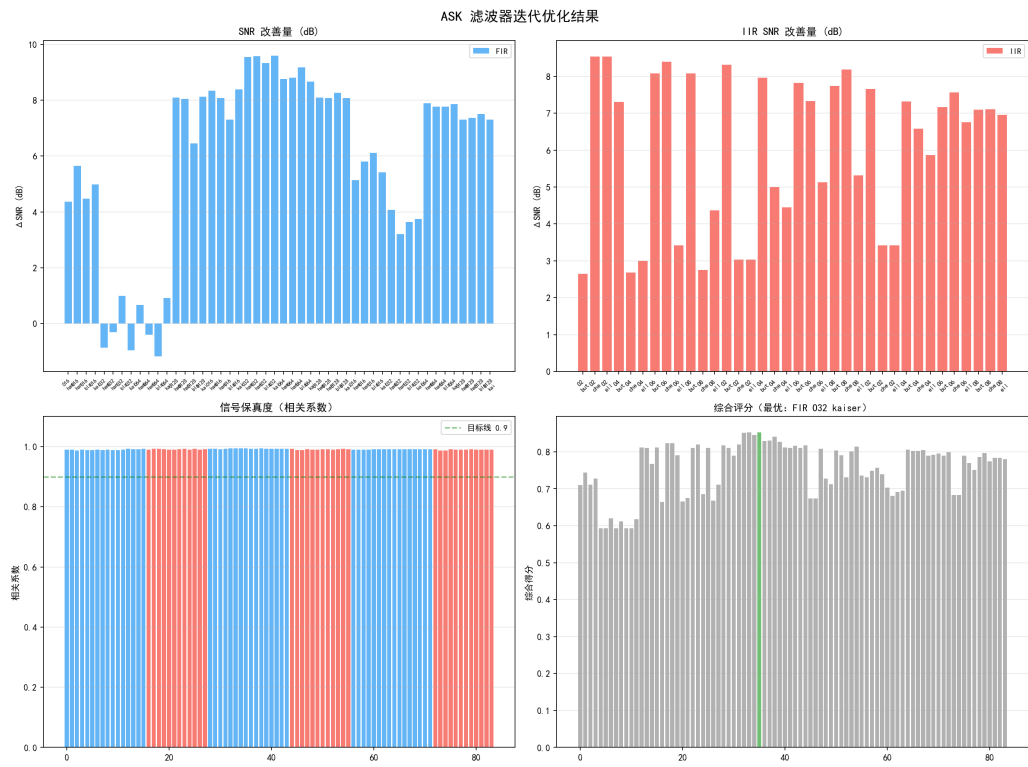


图 9: ASK 滤波器参数迭代优化（84 种组合扫描：SNR 改善/保真度/综合评分）

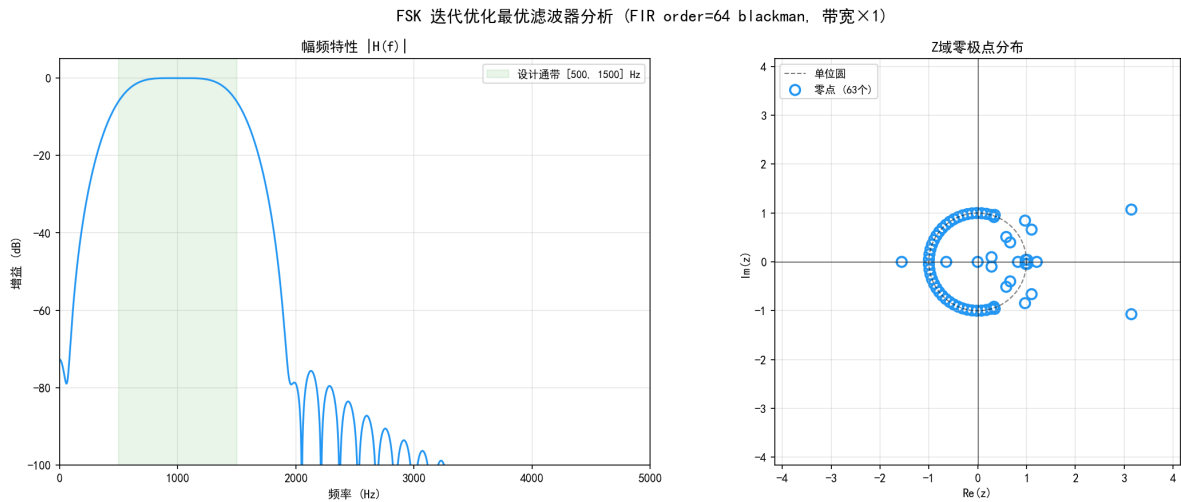


图 10: FSK 迭代优化最优滤波器分析（幅频特性 + 设计通带 / Z 域零极点分布）

滤波前后波形对比（图 11）显示 FIR 滤波有效抑制带外噪声，输出波形与纯净信号高度吻合。

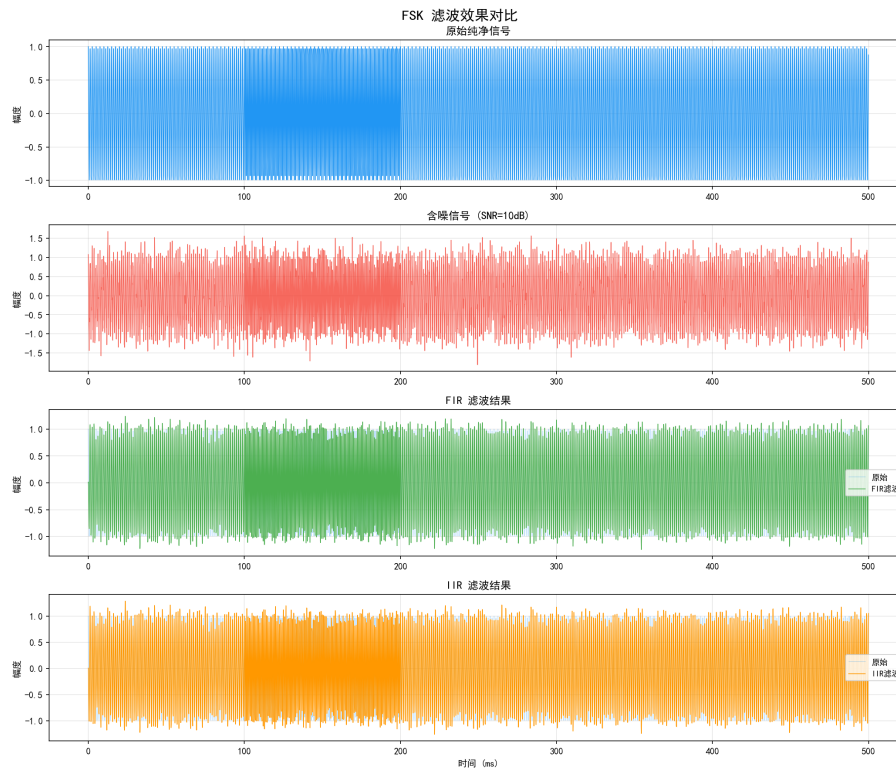


图 11: FSK 滤波前后波形对比（纯净/含噪/FIR/IIR）

4.4 NLMS 自适应滤波

图 12 为 ASK 的 NLMS 详细分析：MSE 收敛曲线（对数坐标）、不同步长性能对比、权值演化与学习曲线。权值 $w[0]$ 快速收敛到 1（其余趋于 0），与维纳解 $[1, 0, \dots, 0]$ 吻合，验证了 ANC 参考信道设计的正确性。表 6 的步长扫描呈现典型折中： $\mu = 0.01$ 稳态最优（ $\Delta\text{SNR} = +9.41 \text{ dB}$ ）， $\mu \geq 0.5$ 因梯度噪声过大反而恶化。三种调制的最优步长均为 0.01，最终 NLMS 性能与 FIR 迭代最优相当（FSK 场景 NLMS +9.02 dB 甚至优于 FIR 迭代最优的 +8.10 dB）。

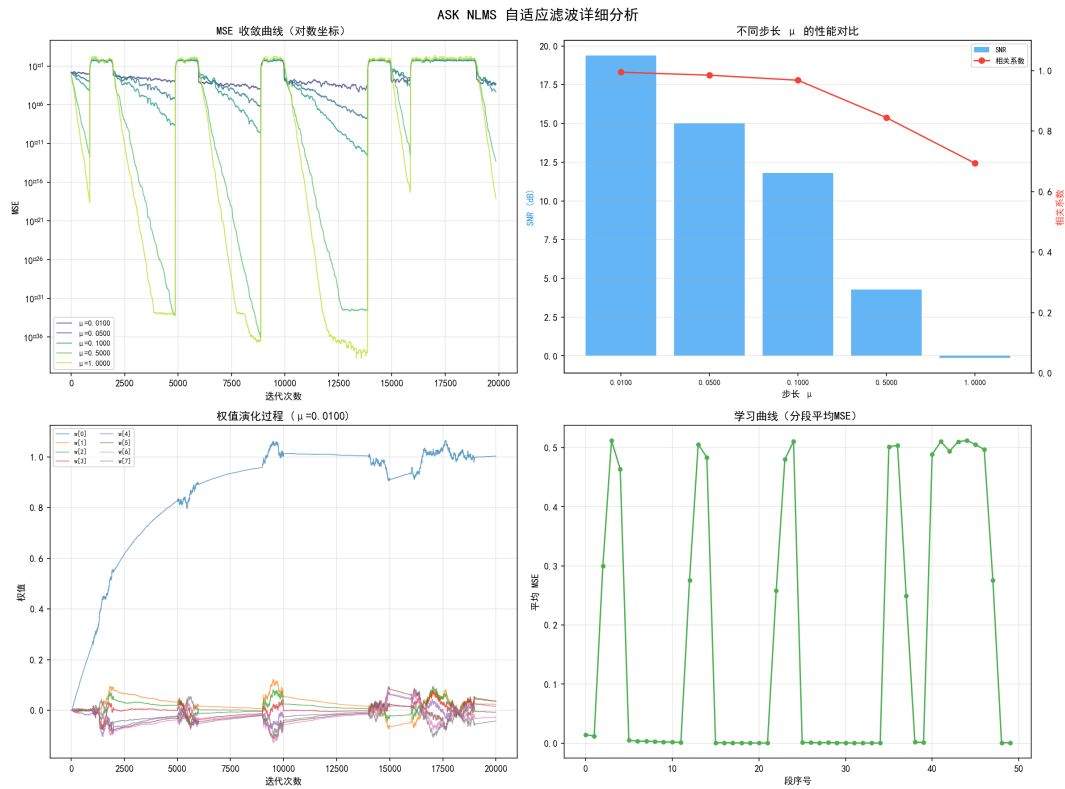


图 12: ASK 的 NLMS 自适应滤波详细分析 (收敛曲线/步长对比/权值演化/学习曲线)

表 6: NLMS 步长扫描结果 (SNR=10 dB)

调制	μ	输出 SNR (dB)	Δ SNR (dB)	相关系数	稳态 MSE
ASK	0.01	19.39	+9.41	0.9943	0.2523
ASK	0.05	15.01	+5.03	0.9845	0.2622
ASK	0.10	11.79	+1.81	0.9679	0.2740
ASK	0.50	4.27	-5.71	0.8443	0.3528
FSK	0.01	19.04	+9.02	0.9938	0.4918
PSK	0.01	18.96	+9.00	0.9937	0.5025

4.5 调制方式自动识别

识别结果 (图 14) 为 3/3 全部正确: ASK (置信度 73.7%, 依据: 包络 CV=1.186、频谱单峰集中、Z 域谱线展宽 $r = 0.956$); FSK (52.2%, 双峰间隔 400 Hz、Z 域尖锐谐振 $r = 0.998$); PSK (54.2%, 单峰恒包络窄带特征、 $r = 0.999$)。Z 域证据实际参与了每一例判决, 体现多域联合设计。

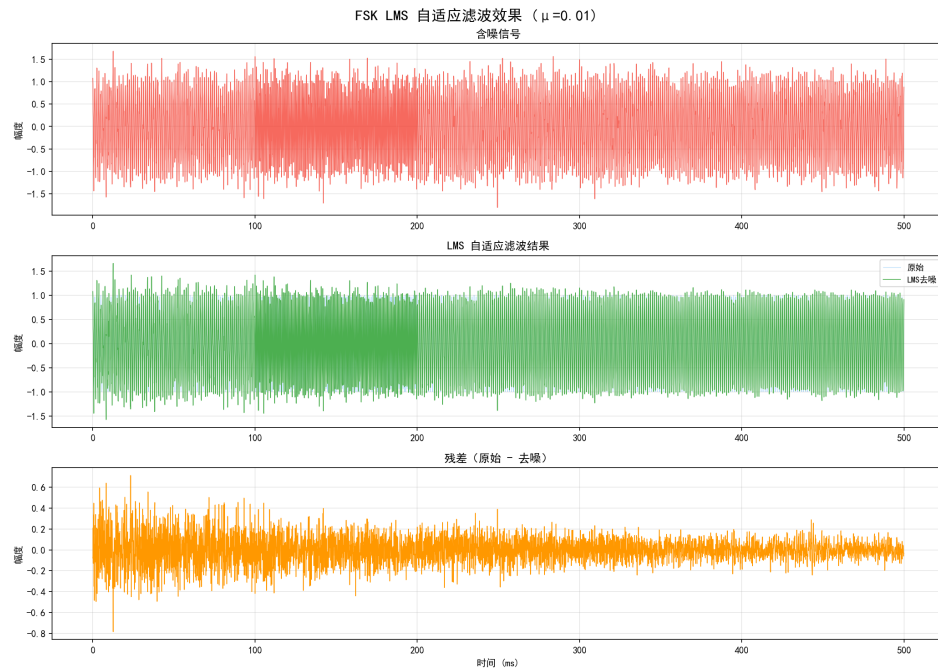


图 13: FSK 的 NLMS 去噪波形对比 (含噪/去噪/残差)

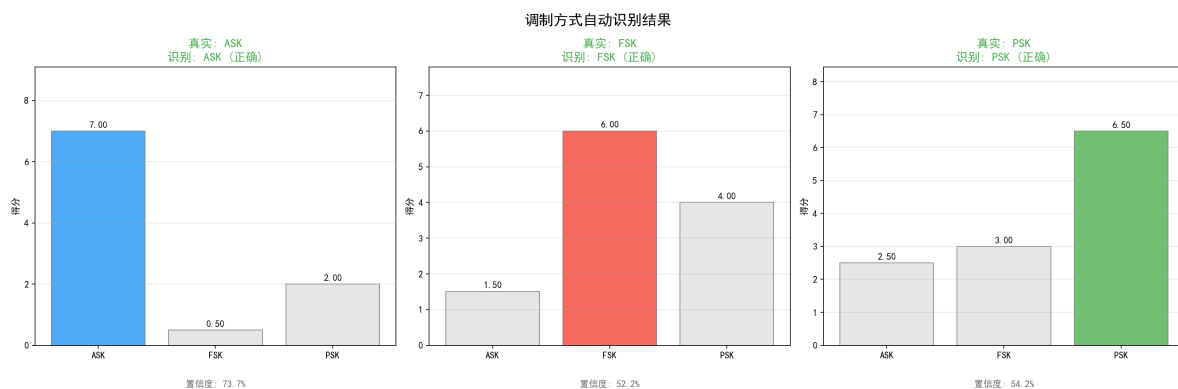


图 14: 调制方式自动识别结果 (SNR=10 dB, 准确率 100%)

4.6 综合性能评估：SNR、保真度、BER 与运算复杂度

表 7 汇总各滤波方法在 10 dB 下的综合性能。要点：

1. FIR 迭代最优取得最大 ΔSNR (9.60 dB)；NLMS 以 128 乘加/样本达到 9.41 dB，接近 FIR；
2. IIR 运算复杂度最低 (34 乘加/样本，仅为 FIR 的 1/4)， ΔSNR 略低 1 dB，适合资源受限场景；
3. 所有滤波方法 BER 均为 0——10 dB 下每符号 1000 点的相关解调处理增益极大，波形级差异在解调级被“消化”，这促使系统将 SNR 扫描扩展到深噪区间以暴露 BER 差异（见下节）。

表 7: 滤波方法综合性能评估 (SNR=10 dB, 乘加/样本为 fltflt 双向计)

调制	方法	Δ SNR (dB)	保真度	BER	乘加/样本	耗时 (ms)
ASK	FIR 最优	+8.75	0.9934	0	130	2.47
ASK	IIR 最优	+7.96	0.9921	0	34	0.00
ASK	FIR 迭代最优	+9.60	0.9945	0	66	0.00
ASK	NLMS 自适应	+9.41	0.9943	0	128	151.6
FSK	FIR 最优	+7.95	0.9921	0	130	2.00
FSK	IIR 最优	+7.28	0.9908	0	34	0.00
FSK	FIR 迭代最优	+8.10	0.9924	0	130	1.00
FSK	NLMS 自适应	+9.02	0.9938	0	128	158.1
PSK	FIR 最优	+8.55	0.9930	0	130	1.00
PSK	IIR 最优	+7.83	0.9917	0	34	0.95
PSK	FIR 迭代最优	+9.27	0.9940	0	66	1.45
PSK	NLMS 自适应	+9.00	0.9937	0	128	158.3

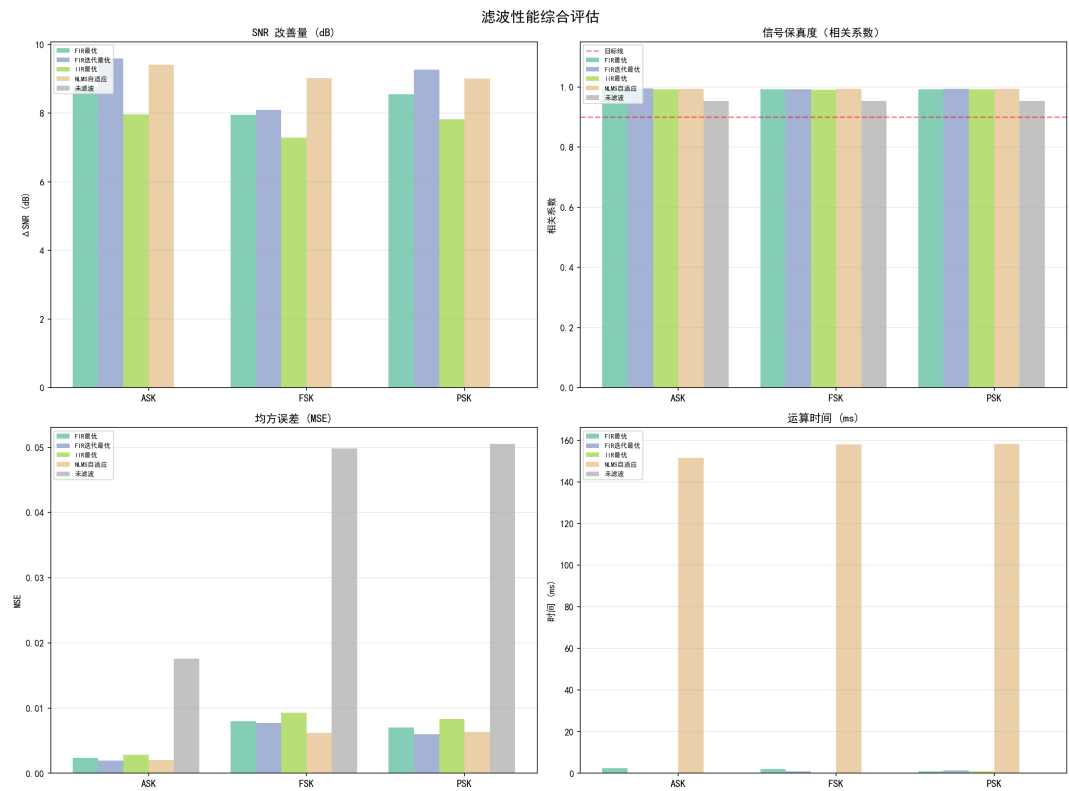


图 15: 滤波性能综合评估 (Δ SNR/保真度/MSE/耗时)

4.7 SNR 扫描蒙特卡洛实验

表 8 与图 16 给出七个 SNR 点（每点 5 次独立试验）的平均性能。深噪区（-25 dB）结果尤为关键：

- **NLMS 展现压倒性优势：** $\Delta\text{SNR} = +23.24\text{ dB}$ （利用噪声参考直接对消）， $\text{BER} = 0.0067$ ，比 FIR（0.23）低 **34 倍**——在极低 SNR 下带外噪声已淹没带内，频率选择性滤波失效，而时域噪声对消仍然有效；
- **FIR 在中低 SNR（-20 ~ -15 dB）稳定优于未滤波：**BER 从 0.123 降至 0.077（-20 dB）、0.093 降至 0.010（-15 dB）；
- **识别准确率随 SNR 单调上升：**-25 dB 时 27%（接近随机），-10 dB 起 93–100%。

表 8: SNR 扫描蒙特卡洛结果（每点 5 次独立试验平均）

SNR(dB)	ΔSNR FIR(dB)	ΔSNR IIR(dB)	ΔSNR NLMS(dB)	BER 未滤波	BER FIR	BER NLMS	识别 准确率
-25	8.97	8.11	23.24	0.2000	0.2300	0.0067	27%
-20	8.96	8.08	21.42	0.1233	0.0767	0.0100	40%
-15	8.93	8.05	19.26	0.0933	0.0100	0.0067	67%
-10	8.96	8.09	18.22	0	0	0.0033	93%
0	8.94	8.08	13.62	0	0	0	100%
10	8.47	7.72	9.00	0	0	0	100%
20	5.53	5.30	2.96	0	0	0	100%

4.8 单一域 vs 多域联合消融实验

表 9 为消融实验结果，直接回答题目“对比单一域处理与多域联合处理的优势”的要求：

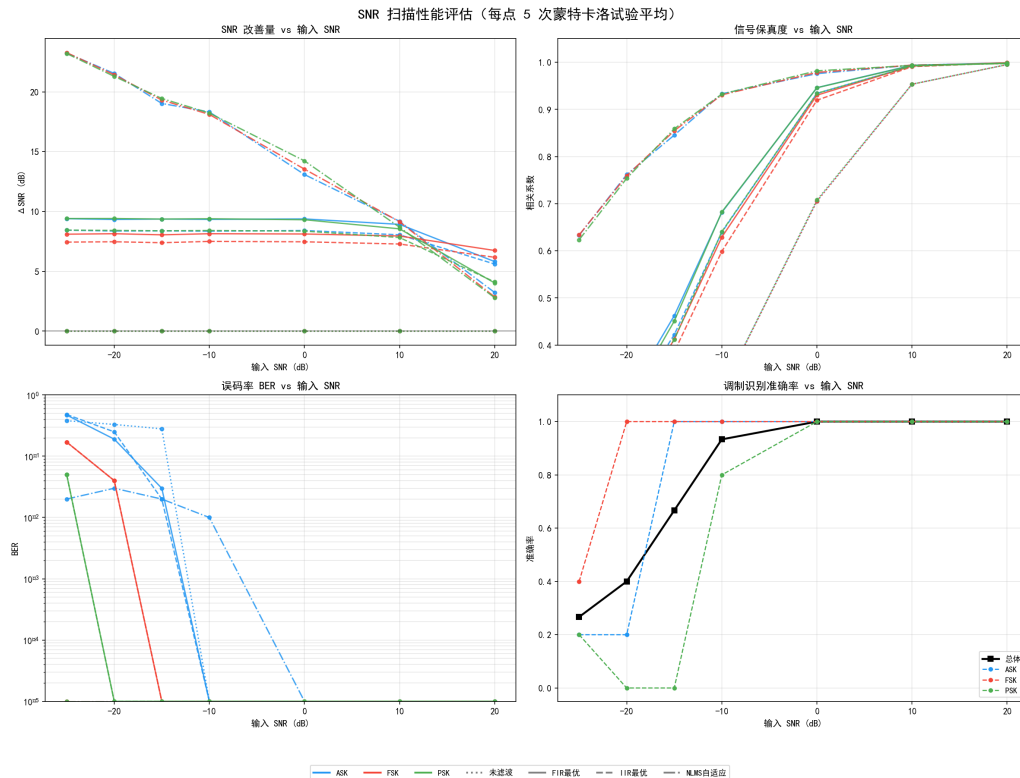


图 16: SNR 扫描四联图 (Δ SNR/保真度/BER/识别准确率, 颜色 = 调制方式, 线型 = 滤波方法)

表 9: 单一域 vs 多域联合识别准确率消融 (FIR 滤波后信号)

SNR(dB)	仅时域	仅频域	仅 Z 域	多域联合
-25	33%	47%	33%	47%
-20	33%	60%	47%	67%
-15	33%	67%	33%	47%
-10	47%	67%	40%	53%
0	67%	67%	53%	100%
10	67%	67%	40%	100%
20	67%	67%	47%	93%

核心发现:

1. 各单域均存在系统性盲区: $\text{SNR} \geq 0$ dB 时三个单域的准确率上限均为 67%——时域无法区分 FSK/PSK (均恒包络)、频域无法区分 ASK/PSK (均单峰)、Z 域无法区分 FSK/PSK (均尖锐谐振), 但三者盲区互不重叠;
2. 多域联合消除盲区: $\text{SNR} \geq 0$ dB 平均准确率 97.8%, 相对最佳单域增益 +31.1 个百分点;

3. 深噪区融合无法创造信息：SNR < -10 dB 时各域特征本身失效，准确率共同退化——多域融合的前提是各域特征仍有效。

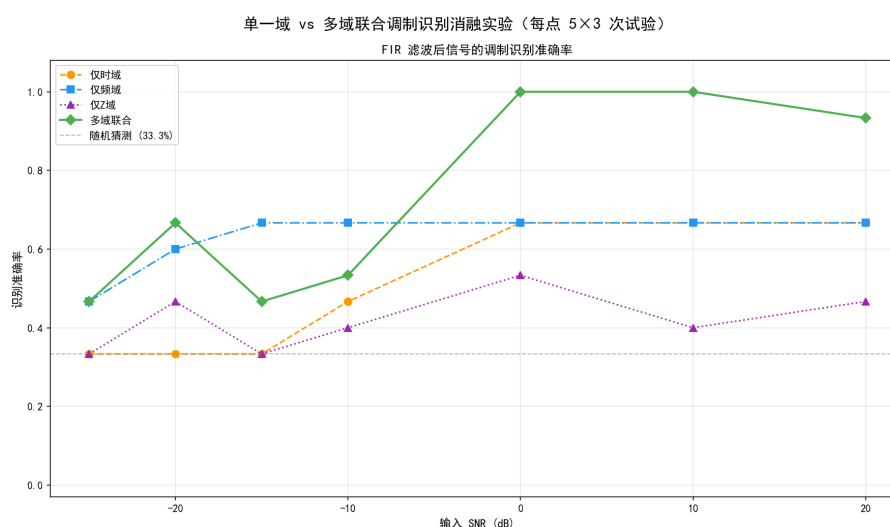


图 17: 单一域 vs 多域联合识别准确率消融曲线

5 创新拓展与总结

5.1 创新点

1. **NLMS 归一化自适应滤波**：在题目要求的“可自主尝试简单自适应滤波算法”基础上，针对标准 LMS 在白噪声参考下收敛慢的缺陷，实现步长与输入功率解耦的 NLMS，并通过候选步长按条件择优策略，使深噪区（-25 dB） Δ SNR 达 +23.2 dB、BER 较 FIR 降低 34 倍；
2. **解调级 BER 评估体系**：突破波形级指标局限，实现三种调制的完整解调链路（包络检波/相关/相干），揭示“滤波改善波形但 10 dB 下 BER 均为零”的处理增益现象，并将扫描扩展至 -25 dB 暴露方法间差异；
3. **Z 域特征融入识别**：以 AR 主极点半径刻画谱线锐度，使“多域联合”名实相符；消融实验定量证明各单域盲区互补、多域增益 +31.1%；
4. **三维参数网格迭代优化**：阶数 × 窗函数 × 截止频率 84 组合/调制的自动扫描，以保真度与归一化 Δ SNR 加权评分自动选型。

5.2 各变换工具的分工与协同

- **时域卷积**：低通平滑压制宽带噪声、提取包络/瞬时频率等调制特征，是识别的“第一道工序”；

- **DTFT**：连续谱形与带宽观察，指导带通滤波器截止频率设计；
- **DFT**：离散谱线与峰值检测，直接产出识别用的频域特征（双峰结构、谱集中度）；
- **Z 变换**：零极点将“谱线锐度”映射为极点半径这一系统级特征，并支撑滤波器零极点形态分析；
- **滤波器**：FIR 线性相位保形、IIR 低复杂度高效、NLMS 时域对消，三者按场景互补。

5.3 局限与展望

（1）识别规则阈值与当前参数（ F_s 、符号率、频点间隔）耦合，迁移到其他参数体制需重新标定；（2）NLMS 依赖理想噪声参考，实际系统中参考信道噪声与信道噪声仅部分相关，可进一步研究延迟参考 ANC 与自适应谱线增强器（ALE）；（3）深噪区识别准确率仍低，可引入高阶累积量或深度学习分类器；（4）蒙特卡洛试验次数可进一步增大以收窄 BER 置信区间。

A 附录：关键程序代码

以下代码直接引自工程源文件，与提交的运行结果一一对应。完整工程含 7 个源文件约 3600 行，此处收录核心实现。

A.1 全局参数配置（config.py，节选）

```

1 # ===== 采样与时间参数 =====
2 FS = 10000          # 采样率 (Hz)
3 DURATION = 0.1      # 单个符号持续时间 (s)
4 N_BITS = 20         # 信息比特数
5 SAMPLES_PER_BIT = int(FS * DURATION) # 每比特采样点数
6
7 # ===== ASK 调制参数 =====
8 FC_ASK = 1000       # ASK 载波频率 (Hz)
9 ASK_AMPLITUDE_1 = 1.0 # ASK bit=1 幅度
10 ASK_AMPLITUDE_0 = 0.0 # ASK bit=0 幅度 (OOK)
11
12 # ===== FSK 调制参数 =====
13 FC_FSK_F1 = 800     # FSK bit=0 对应频率 (Hz)
14 FC_FSK_F2 = 1200    # FSK bit=1 对应频率 (Hz)
15
16 # ===== PSK 调制参数 =====
17 FC_PSK = 1000       # PSK 载波频率 (Hz)
18 PSK_PHASE_0 = 0     # PSK bit=0 对应相位 (rad)
19 PSK_PHASE_1 = np.pi # PSK bit=1 对应相位 (rad)
20
21 # ===== 信道参数 =====
22 SNR_DB = 10         # 默认信噪比 (dB)

```

```

23 # SNR 扫描范围：包含深噪区间 (-25 ~ -15 dB) 用于体现 BER 与识别性能差异
24 SNR_RANGE = [-25, -20, -15, -10, 0, 10, 20]
25
26 # ===== 滤波器参数 =====
27 # FIR 参数
28 FIR_ORDERS = [16, 32, 64, 128]
29 FIR_WINDOWS = ['hamming', 'hann', 'blackman', 'kaiser']
30 FIR_DEFAULT_ORDER = 64
31 FIR_DEFAULT_WINDOW = 'hamming'
32
33 # IIR 参数
34 IIR_ORDERS = [2, 4, 6, 8]
35 IIR_TYPES = ['butter', 'cheby1', 'ellip']
36 IIR_DEFAULT_ORDER = 4
37 IIR_DEFAULT_TYPE = 'butter'
38
39 # 带通滤波器截止频率 (Hz)
40 BANDPASS_LOW_ASK = 600
41 BANDPASS_HIGH_ASK = 1400
42 BANDPASS_LOW_FSK = 500
43 BANDPASS_HIGH_FSK = 1500
44 BANDPASS_LOW_PSK = 600
45 BANDPASS_HIGH_PSK = 1400
46
47 # 迭代优化中截止频率的缩放系数扫描 (相对名义带宽)
48 CUTOFF_SCALES = [0.8, 1.0, 1.2]
49
50 # ===== LMS/NLMS 自适应滤波参数 =====
51 LMS_ORDER = 32      # LMS 滤波器阶数
52 LMS_MU = 0.01       # LMS 步长 (学习率)
53 LMS_MU_RANGE = [0.001, 0.005, 0.01, 0.05, 0.1] # 步长扫描范围
54
55 # NLMS 归一化 LMS: 步长与输入功率无关, (0,2]
56 NLMS_ORDER = 32     # NLMS 滤波器阶数
57 NLMS_MU = 0.02      # 默认归一化步长 (小步长降低稳态失调)
58 NLMS_MU_RANGE = [0.01, 0.05, 0.1, 0.5, 1.0] # 归一化步长扫描范围
59 NLMS_SWEEP_MUS = [0.01, 0.05, 0.1, 0.5] # SNR 扫描中按条件择优的候选步长
60
61 # ===== SNR 扫描实验参数 =====
62 SWEEP_TRIALS = 5     # 每个 SNR 点的蒙特卡洛试验次数

```

A.2 信号生成 (utils/generator.py)

```

1 def generate_binary_sequence(n_bits, seed=None):
2     """
3     生成随机二进制比特流
4
5     Parameters
6     -----
7     n_bits : int
8         比特数量
9     seed : int, optional
10         随机种子 (用于可复现性)
11
12     Returns

```



```

13  -----
14  bits : ndarray
15      0/1 二进制序列
16  """
17  if seed is not None:
18      rng = np.random.RandomState(seed)
19  else:
20      rng = np.random.RandomState(config.RANDOM_SEED)
21  return rng.randint(0, 2, n_bits)
22
23
24  def generate_ask(bits, fs=None, fc=None, A1=None, A0=None, samples_per_bit=None):
25      """
26      生成 2-ASK (OOK) 调制信号
27
28      原理:  $s(n) = A(m) * \cos(2 * \pi * fc * n * Ts)$ 
29      其中  $A(m) = A1$  if bit=1,  $A0$  if bit=0
30
31      Parameters
32      -----
33      bits : ndarray
34          二进制比特序列
35      fs : float
36          采样率
37      fc : float
38          载波频率
39      A1, A0 : float
40          bit=1/0 对应的幅度
41      samples_per_bit : int
42          每比特采样点数
43
44      Returns
45      -----
46      signal : ndarray
47          ASK 调制信号
48      t : ndarray
49          时间轴
50      """
51      fs = fs or config.FS
52      fc = fc or config.FC_ASK
53      A1 = A1 if A1 is not None else config.ASK_AMPLITUDE_1
54      A0 = A0 if A0 is not None else config.ASK_AMPLITUDE_0
55      samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
56
57      n_total = len(bits) * samples_per_bit
58      t = np.arange(n_total) / fs
59      signal = np.zeros(n_total)
60
61      for i, bit in enumerate(bits):
62          start = i * samples_per_bit
63          end = (i + 1) * samples_per_bit
64          amplitude = A1 if bit == 1 else A0
65          t_seg = np.arange(samples_per_bit) / fs
66          signal[start:end] = amplitude * np.cos(2 * np.pi * fc * t_seg)
67
68      return signal, t

```

```

69
70
71 def generate_fsk(bits, fs=None, f1=None, f2=None, samples_per_bit=None):
72     """
73     生成 2-FSK 调制信号
74
75     原理: bit=0 → cos(2 * f1 * n * Ts)
76           bit=1 → cos(2 * f2 * n * Ts)
77
78     Parameters
79     -----
80     bits : ndarray
81         二进制比特序列
82     fs : float
83         采样率
84     f1, f2 : float
85         bit=0/1 对应的载波频率
86     samples_per_bit : int
87         每比特采样点数
88
89     Returns
90     -----
91     signal : ndarray
92         FSK 调制信号
93     t : ndarray
94         时间轴
95     """
96     fs = fs or config.FS
97     f1 = f1 or config.FC_FSK_F1
98     f2 = f2 or config.FC_FSK_F2
99     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
100
101     n_total = len(bits) * samples_per_bit
102     t = np.arange(n_total) / fs
103     signal = np.zeros(n_total)
104
105     for i, bit in enumerate(bits):
106         start = i * samples_per_bit
107         end = (i + 1) * samples_per_bit
108         freq = f2 if bit == 1 else f1
109         t_seg = np.arange(samples_per_bit) / fs
110         signal[start:end] = np.cos(2 * np.pi * freq * t_seg)
111
112     return signal, t
113
114
115 def generate_psk(bits, fs=None, fc=None, phase_0=None, phase_1=None, samples_per_bit=None):
116     """
117     生成 2-PSK (BPSK) 调制信号
118
119     原理: bit=0 → cos(2 * fc * n * Ts + 0)
120           bit=1 → cos(2 * fc * n * Ts + 1)
121
122     Parameters
123     -----
124     bits : ndarray

```

```

125     二进制比特序列
126     fs : float
127     采样率
128     fc : float
129     载波频率
130     phase_0, phase_1 : float
131     bit=0/1 对应的相位 (rad)
132     samples_per_bit : int
133     每比特采样点数
134
135     Returns
136     -----
137     signal : ndarray
138         PSK 调制信号
139     t : ndarray
140         时间轴
141
142     """
143     fs = fs or config.FS
144     fc = fc or config.FC_PSK
145     phase_0 = phase_0 if phase_0 is not None else config.PSK_PHASE_0
146     phase_1 = phase_1 if phase_1 is not None else config.PSK_PHASE_1
147     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
148
149     n_total = len(bits) * samples_per_bit
150     t = np.arange(n_total) / fs
151     signal = np.zeros(n_total)
152
153     for i, bit in enumerate(bits):
154         start = i * samples_per_bit
155         end = (i + 1) * samples_per_bit
156         phase = phase_1 if bit == 1 else phase_0
157         t_seg = np.arange(samples_per_bit) / fs
158         signal[start:end] = np.cos(2 * np.pi * fc * t_seg + phase)
159
160     return signal, t
161
162 def add_awgn(signal, snr_db=None):
163     """
164     叠加上加性高斯白噪声 (AWGN)
165
166     原理: 噪声功率 = 信号功率 / (10^(SNR/10))
167
168     Parameters
169     -----
170     signal : ndarray
171         纯净信号
172     snr_db : float
173         信噪比 (dB)
174
175     Returns
176     -----
177     noisy_signal : ndarray
178         含噪信号
179     noise : ndarray
180         噪声分量 (便于后续分析)

```

```

181     """
182     snr_db = snr_db if snr_db is not None else config.SNR_DB
183
184     # 计算信号功率
185     signal_power = np.mean(signal ** 2)
186
187     # 根据 SNR 计算噪声功率
188     snr_linear = 10 ** (snr_db / 10)
189     noise_power = signal_power / snr_linear
190
191     # 生成高斯白噪声
192     noise = np.sqrt(noise_power) * np.random.randn(len(signal))
193
194     noisy_signal = signal + noise
195     return noisy_signal, noise

```

A.3 多域分析核心算法（utils/analyzer.py，节选）

时域特征与卷积平滑：

```

1 def convolution_smooth(signal_data, kernel_size=11):
2     """
3     时域卷积平滑
4     使用矩形窗(均值滤波器)对信号进行卷积平滑
5
6     Parameters
7     -----
8     signal_data : ndarray
9         输入信号
10    kernel_size : int
11        卷积核大小（奇数）
12
13    Returns
14    -----
15    smoothed : ndarray
16        平滑后的信号（same 模式）
17    """
18    kernel = np.ones(kernel_size) / kernel_size
19    smoothed = np.convolve(signal_data, kernel, mode='same')
20    return smoothed
21
22
23 def compute_envelope(signal_data):
24     """
25     计算信号包络（利用希尔伯特变换）
26
27     Parameters
28     -----
29     signal_data : ndarray
30        输入信号
31
32    Returns
33    -----
34    envelope : ndarray
35        信号包络

```

```

36     """
37     analytic = sig.hilbert(signal_data)
38     envelope = np.abs(analytic)
39     return envelope
40
41
42 def time_domain_features(signal_data):
43     """
44     提取时域统计特征
45
46     Returns
47     -----
48     features : dict
49         包含均值、方差、峰值、包络方差等统计量
50     """
51     envelope = compute_envelope(signal_data)
52     features = {
53         'mean': np.mean(signal_data),
54         'variance': np.var(signal_data),
55         'rms': np.sqrt(np.mean(signal_data ** 2)),
56         'peak': np.max(np.abs(signal_data)),
57         'envelope_mean': np.mean(envelope),
58         'envelope_var': np.var(envelope),
59         'zero_crossings': np.sum(np.diff(np.sign(signal_data)) != 0)
60     }
61     return features

```

DFT/DTFT/Z 域零极点:

```

1 def compute_dft(signal_data, N=None):
2     """
3     计算 N 点 DFT 离散频谱
4
5     Parameters
6     -----
7     signal_data : ndarray
8         输入信号
9     N : int
10        DFT 点数 (默认为信号长度)
11
12    Returns
13    -----
14    freqs : ndarray
15        频率轴 (Hz)
16    magnitude : ndarray
17        幅度谱
18    phase : ndarray
19        相位谱
20    """
21    if N is None:
22        N = len(signal_data)
23
24    # 使用 FFT 高效计算 DFT
25    X = np.fft.fft(signal_data, n=N)
26    freqs = np.fft.fftfreq(N, d=1.0 / config.FS)
27
28    # 只取正频率部分

```

```

29     positive_mask = freqs >= 0
30     freqs = freqs[positive_mask]
31     X = X[positive_mask]
32
33     # 单边谱归一化: DC 与 Nyquist 分量不加倍, 其余加倍
34     magnitude = np.abs(X) / N
35     double_mask = (freqs > 0) & (freqs < config.FS / 2)
36     magnitude[double_mask] *= 2
37     phase = np.angle(X)
38
39     return freqs, magnitude, phase
40
41
42 def compute_dtft(signal_data, n_points=4096):
43     """
44     计算 DTFT 连续频谱 (通过密集 FFT 逼近)
45
46     DTFT 定义:  $X(\omega) = \sum x[n] * e^{-j\omega n}$ 
47     通过零填充的 FFT 在密集频率点上近似 DTFT
48
49     Parameters
50     -----
51     signal_data : ndarray
52         输入信号
53     n_points : int
54         频率采样点数 (越大越逼近连续)
55
56     Returns
57     -----
58     omega : ndarray
59         数字角频率 [0,  $\pi$ ]
60     magnitude : ndarray
61         幅度谱  $|X(\omega)|$ 
62     phase : ndarray
63         相位谱  $\angle X(\omega)$ 
64     """
65     # 零填充到 n_points 实现密集频率采样
66     N = max(n_points, len(signal_data))
67     X = np.fft.fft(signal_data, n=N)
68
69     # 取正频率部分 [0,  $\pi$ ]
70     half = N // 2
71     omega = np.linspace(0, np.pi, half)
72     X_half = X[:half]
73
74     magnitude = np.abs(X_half)
75     phase = np.angle(X_half)
76
77     return omega, magnitude, phase
78
79
80 def compute_zpk(b, a):
81     """
82     Z 变换零极点计算
83
84     Parameters

```

```

85  -----
86  b : ndarray
87      系统函数分子多项式系数
88  a : ndarray
89      系统函数分母多项式系数
90
91  Returns
92  -----
93  zeros : ndarray
94      零点
95  poles : ndarray
96      极点
97  gain : float
98      增益
99  """
100  zeros, poles, gain = sig.tf2zpk(b, a)
101  return zeros, poles, gain
102
103
104  def signal_to_zpk(signal_data, order=10):
105      """
106      对信号进行 AR 模型拟合，提取 Z 域零极点
107      用于分析不同调制方式的 Z 域特征
108
109      Parameters
110      -----
111      signal_data : ndarray
112          输入信号（取一段）
113      order : int
114          AR 模型阶数
115
116      Returns
117      -----
118      zeros : ndarray
119          零点
120      poles : ndarray
121          极点
122      gain : float
123          增益
124      """
125      # 使用 Yule-Walker 方法拟合 AR 模型
126      # 计算自相关
127      seg = signal_data[:min(1000, len(signal_data))]
128      autocorr = np.correlate(seg, seg, mode='full')
129      autocorr = autocorr[len(autocorr)//2:]
130
131      # 信号过弱（如 ASK 静音段）时自相关无意义，返回简单系统
132      if autocorr[0] < 1e-12:
133          b = np.array([1.0])
134          a = np.array([1.0, -0.5])
135          return sig.tf2zpk(b, a)
136
137      autocorr = autocorr / autocorr[0] # 归一化
138
139      # Levinson-Durbin 递推求 AR 系数
140      r = autocorr[1:order+1]

```

```

141 R = np.zeros((order, order))
142 for i in range(order):
143     for j in range(order):
144         R[i, j] = autocorr[abs(i - j)]
145
146 try:
147     a_coeffs = np.linalg.solve(R, r)
148     a = np.concatenate(([1], -a_coeffs))
149     b = np.array([1.0])
150     zeros, poles, gain = sig.tf2zpk(b, a)
151 except np.linalg.LinAlgError:
152     # 如果矩阵奇异，返回简单结果
153     b = np.array([1.0])
154     a = np.array([1.0, -0.5])
155     zeros, poles, gain = sig.tf2zpk(b, a)
156
157 return zeros, poles, gain

```

A.4 滤波器设计与自适应滤波（utils/filter_design.py，节选）

FIR/IIR 设计与零相位滤波：

```

1 def design_fir(order, cutoff_low, cutoff_high, fs=None, window='hamming'):
2     """
3     FIR 带通滤波器设计（窗函数法）
4
5     Parameters
6     -----
7     order : int
8         滤波器阶数
9     cutoff_low, cutoff_high : float
10        带通截止频率（Hz）
11     fs : float
12        采样率
13     window : str
14        窗函数类型（'hamming', 'hann', 'blackman', 'kaiser'）
15
16     Returns
17     -----
18     b : ndarray
19        FIR 滤波器系数
20     a : ndarray
21        固定为 [1]（FIR无反馈）
22     """
23     fs = fs or config.FS
24     nyq = fs / 2.0
25
26     # 归一化截止频率
27     low = cutoff_low / nyq
28     high = cutoff_high / nyq
29
30     # 确保频率在有效范围内
31     low = max(0.001, min(low, 0.999))
32     high = max(low + 0.001, min(high, 0.999))
33

```



```

34 # 窗函数法设计 FIR
35 if window == 'kaiser':
36     b = sig.firwin(order + 1, [low, high], pass_zero=False,
37                     window=('kaiser', 5.0))
38 else:
39     b = sig.firwin(order + 1, [low, high], pass_zero=False,
40                     window=window)
41
42 a = np.array([1.0])
43 return b, a
44
45
46 def design_iir(order, cutoff_low, cutoff_high, fs=None, ftype='butter'):
47     """
48     IIR 带通滤波器设计
49
50     Parameters
51     -----
52     order : int
53         滤波器阶数
54     cutoff_low, cutoff_high : float
55         带通截止频率 (Hz)
56     fs : float
57         采样率
58     ftype : str
59         滤波器类型 ('butter', 'cheby1', 'ellip')
60
61     Returns
62     -----
63     b : ndarray
64         分子多项式系数
65     a : ndarray
66         分母多项式系数
67     """
68     fs = fs or config.FS
69     nyq = fs / 2.0
70
71     low = cutoff_low / nyq
72     high = cutoff_high / nyq
73
74     low = max(0.001, min(low, 0.999))
75     high = max(low + 0.001, min(high, 0.999))
76
77     if ftype == 'butter':
78         b, a = sig.butter(order, [low, high], btype='band')
79     elif ftype == 'cheby1':
80         b, a = sig.cheby1(order, 1, [low, high], btype='band') # 1dB纹波
81     elif ftype == 'ellip':
82         b, a = sig.ellip(order, 1, 40, [low, high], btype='band') # 1dB纹波, 40dB阻带衰减
83     else:
84         raise ValueError(f"不支持的滤波器类型: {ftype}")
85
86     return b, a
87
88
89 def apply_filter(b, a, signal_data):

```



```

27         fir_orders=None, fir_windows=None,
28         iir_orders=None, iir_types=None,
29         cutoff_scales=None):
30
31     """
32     迭代优化滤波器参数
33     遍历阶数、窗函数/类型、截止频率缩放组合，选择最优方案
34
35     Returns
36     -----
37     results : list of dict
38         每种组合的评价结果
39     best_result : dict
40         最优结果
41     """
42     fs = fs or config.FS
43     fir_orders = fir_orders or config.FIR_ORDERS
44     fir_windows = fir_windows or config.FIR_WINDOWS
45     iir_orders = iir_orders or config.IIR_ORDERS
46     iir_types = iir_types or config.IIR_TYPES
47     cutoff_scales = cutoff_scales or config.CUTOFF_SCALES
48
49     nyq = fs / 2.0
50     snr_in_ref = compute_snr(clean_signal, noisy_signal)
51
52     results = []
53
54     # 遍历截止频率缩放 (调整带宽 → 改变 Z 域零点位置)
55     for scale in cutoff_scales:
56         eff_low = max(50.0, min(cutoff_low * scale, nyq - 100))
57         eff_high = max(eff_low + 100, min(cutoff_high * scale, nyq - 50))
58
59         # FIR 参数扫描
60         for order in fir_orders:
61             for window in fir_windows:
62                 try:
63                     t_start = time.time()
64                     b, a = design_fir(order, eff_low, eff_high, fs, window)
65                     filtered = apply_filter(b, a, noisy_signal)
66                     t_elapsed = time.time() - t_start
67
68                     snr_out = compute_snr(clean_signal, filtered)
69                     corr = compute_correlation(clean_signal, filtered)
70
71                     results.append({
72                         'type': 'FIR',
73                         'order': order,
74                         'window': window,
75                         'cutoff_scale': scale,
76                         'cutoff': (eff_low, eff_high),
77                         'snr_improvement': snr_out - snr_in_ref,
78                         'snr_out': snr_out,
79                         'correlation': corr,
80                         'time': t_elapsed,
81                         'b': b,
82                         'a': a,
83                         'filtered': filtered,

```

```

83         # 综合评分: 保真度 0.6 + 归一化SNR改善 0.4
84         'score': corr * 0.6 + min(max(snr_out - snr_in_ref, 0.0), 15.0) / 15.0 * 0.4
85     })
86     except Exception as e:
87         print(f" [跳过] FIR x{scale:g} order={order}, window={window}: {e}")
88
89     # IIR 参数扫描
90     for order in iir_orders:
91         for ftype in iir_types:
92             try:
93                 t_start = time.time()
94                 b, a = design_iir(order, eff_low, eff_high, fs, ftype)
95                 filtered = apply_filter(b, a, noisy_signal)
96                 t_elapsed = time.time() - t_start
97
98                 snr_out = compute_snr(clean_signal, filtered)
99                 corr = compute_correlation(clean_signal, filtered)
100
101                 results.append({
102                     'type': 'IIR',
103                     'order': order,
104                     'window': ftype,
105                     'cutoff_scale': scale,
106                     'cutoff': (eff_low, eff_high),
107                     'snr_improvement': snr_out - snr_in_ref,
108                     'snr_out': snr_out,
109                     'correlation': corr,
110                     'time': t_elapsed,
111                     'b': b,
112                     'a': a,
113                     'filtered': filtered,
114                     'score': corr * 0.6 + min(max(snr_out - snr_in_ref, 0.0), 15.0) / 15.0 * 0.4
115                 })
116             except Exception as e:
117                 print(f" [跳过] IIR x{scale:g} order={order}, type={ftype}: {e}")
118
119     # 选出最优结果
120     if results:
121         best_result = max(results, key=lambda x: x['score'])
122     else:
123         best_result = None
124
125     return results, best_result

```

LMS/NLMS 自适应滤波:

```

1  def lms_filter(desired, input_signal, mu=None, order=None, algorithm='lms'):
2      """
3      自适应滤波算法 (LMS / NLMS)
4
5      LMS:  $w(n+1) = w(n) + 2 \cdot e(n) \cdot x(n)$ 
6           步长受输入功率约束 ( $\mu_{\max} = 1/(3 \cdot \text{order} \cdot P_x)$ ), 收敛慢
7      NLMS:  $w(n+1) = w(n) + e(n) \cdot x(n) / (\|x(n)\|^2 + \epsilon)$ 
8           归一化步长与输入功率无关, (0,2], 收敛快且稳定
9
10     Parameters
11     -----

```

```

12 desired : ndarray
13     期望信号 (参考信号/含噪信号)
14 input_signal : ndarray
15     输入信号 (噪声参考)
16 mu : float
17     步长 (学习率)
18 order : int
19     滤波器阶数
20 algorithm : str
21     'lms' 标准 LMS; 'nlms' 归一化 LMS
22
23 Returns
24 -----
25 output : ndarray
26     滤波输出 (估计的噪声)
27 error : ndarray
28     误差信号 (去噪结果)
29 weights : ndarray
30     最终权值
31 weight_history : ndarray
32     权值演化历史
33 mse_history : ndarray
34     MSE 收敛曲线 (前 order 个样本为 NaN, 未进入迭代)
35 """
36 if algorithm == 'nlms':
37     mu = mu if mu is not None else config.NLMS_MU
38     order = order or config.NLMS_ORDER
39 else:
40     mu = mu if mu is not None else config.LMS_MU
41     order = order or config.LMS_ORDER
42
43 N = len(desired)
44 weights = np.zeros(order)
45 output = np.zeros(N)
46 error = np.zeros(N)
47 weight_history = np.zeros((N, order))
48 mse_history = np.full(N, np.nan)
49
50 # 迭代尚未开始的前 order 个样本直接透传期望信号
51 error[:order] = desired[:order]
52
53 for n in range(order, N):
54     # 输入向量 (含当前样本, 参考信道与时延对齐, 可估计当前噪声)
55     x = input_signal[n:n - order:-1]
56
57     # 滤波输出
58     output[n] = np.dot(weights, x)
59
60     # 误差
61     error[n] = desired[n] - output[n]
62
63     # 权值更新
64     if algorithm == 'nlms':
65         weights = weights + mu * error[n] * x / (np.dot(x, x) + 1e-12)
66     else:
67         weights = weights + 2 * mu * error[n] * x

```

```

68
69     # 记录
70     weight_history[n] = weights.copy()
71     mse_history[n] = error[n] ** 2
72
73     return output, error, weights, weight_history, mse_history
74
75
76 def lms_denoise(noisy_signal, noise_ref=None, mu=None, order=None, algorithm='lms'):
77     """
78     使用自适应滤波进行噪声消除
79
80     当没有噪声参考信号时，使用延迟版本的含噪信号作为参考
81
82     Parameters
83     -----
84     noisy_signal : ndarray
85         含噪信号
86     noise_ref : ndarray, optional
87         噪声参考信号
88     mu, order : float, int
89         自适应滤波参数
90     algorithm : str
91         'lms' 标准 LMS; 'nlms' 归一化 LMS
92
93     Returns
94     -----
95     denoised : ndarray
96         去噪后信号
97     mse_history : ndarray
98         MSE 收敛历史
99     """
100     if noise_ref is None:
101         # 使用延迟版本作为参考
102         delay = order or (config.NLMS_ORDER if algorithm == 'nlms' else config.LMS_ORDER)
103         noise_ref = np.roll(noisy_signal, delay)
104         noise_ref[:delay] = 0
105
106     output, error, weights, weight_history, mse_history = lms_filter(
107         noisy_signal, noise_ref, mu, order, algorithm
108     )
109
110     # 误差信号即为去噪结果
111     return error, mse_history, weights, weight_history

```

A.5 解调与误码率 (utils/demodulator.py)

```

1 def _split_bits(signal_data, samples_per_bit):
2     """将信号按比特切分为二维数组 (n_bits, samples_per_bit)"""
3     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
4     n_bits = len(signal_data) // samples_per_bit
5     return signal_data[:n_bits * samples_per_bit].reshape(n_bits, samples_per_bit)
6
7
8 def demod_ask(signal_data, fs=None, samples_per_bit=None):

```

```

9      """
10     2-ASK (OOK) 包络检波解调
11
12     原理: Hilbert 变换提取包络 → 比特能量积累 → 动态中点阈值判决
13
14     Parameters
15     -----
16     signal_data : ndarray
17         接收信号
18     fs : float
19         采样率
20     samples_per_bit : int
21         每比特采样点数
22
23     Returns
24     -----
25     bits : ndarray
26         解调出的比特序列
27     """
28     fs = fs or config.FS
29     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
30
31     envelope = np.abs(sig.hilbert(signal_data))
32     frames = _split_bits(envelope, samples_per_bit)
33     energies = np.mean(frames ** 2, axis=1)
34
35     # 阈值取能量 10%/90% 分位中点, 对离群帧 (如自适应滤波收敛瞬态) 鲁棒
36     lo = np.percentile(energies, 10)
37     hi = np.percentile(energies, 90)
38     threshold = (lo + hi) / 2
39     return (energies > threshold).astype(int)
40
41
42 def demod_fsk(signal_data, fs=None, f1=None, f2=None, samples_per_bit=None):
43     """
44     2-FSK 相关解调 (匹配滤波器组)
45
46     原理: 每比特分别与两个频率的本地载波作相关运算, 相关值大者判决
47
48     Parameters
49     -----
50     signal_data : ndarray
51         接收信号
52     fs : float
53         采样率
54     f1, f2 : float
55         bit=0/1 对应的载波频率
56     samples_per_bit : int
57         每比特采样点数
58
59     Returns
60     -----
61     bits : ndarray
62         解调出的比特序列
63     """
64     fs = fs or config.FS

```

```

65     f1 = f1 or config.FC_FSK_F1
66     f2 = f2 or config.FC_FSK_F2
67     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
68
69     frames = _split_bits(signal_data, samples_per_bit)
70     t_seg = np.arange(samples_per_bit) / fs
71     ref1 = np.cos(2 * np.pi * f1 * t_seg)
72     ref2 = np.cos(2 * np.pi * f2 * t_seg)
73
74     corr1 = np.abs(frames @ ref1)
75     corr2 = np.abs(frames @ ref2)
76     return (corr2 > corr1).astype(int)
77
78
79 def demod_psk(signal_data, fs=None, fc=None, samples_per_bit=None):
80     """
81     2-PSK (BPSK) 相干解调
82
83     原理：每比特与本地同频同相载波相关，按极性判决
84         0=0 → 相关值为正 → bit=0; 1= → 相关值为负 → bit=1
85
86     Parameters
87     -----
88     signal_data : ndarray
89         接收信号
90     fs : float
91         采样率
92     fc : float
93         载波频率
94     samples_per_bit : int
95         每比特采样点数
96
97     Returns
98     -----
99     bits : ndarray
100         解调出的比特序列
101     """
102     fs = fs or config.FS
103     fc = fc or config.FC_PSK
104     samples_per_bit = samples_per_bit or config.SAMPLES_PER_BIT
105
106     frames = _split_bits(signal_data, samples_per_bit)
107     t_seg = np.arange(samples_per_bit) / fs
108     ref = np.cos(2 * np.pi * fc * t_seg)
109
110     corr = frames @ ref
111     return (corr < 0).astype(int)
112
113
114 DEMOD_FUNCS = {
115     'ASK': demod_ask,
116     'FSK': demod_fsk,
117     'PSK': demod_psk,
118 }
119
120

```



```

121 def compute_ber(true_bits, detected_bits):
122     """
123     计算误码率 (Bit Error Rate)
124
125     Parameters
126     -----
127     true_bits, detected_bits : ndarray
128         真实/解调比特序列
129
130     Returns
131     -----
132     ber : float
133         误码率 [0, 1]
134     """
135     n = min(len(true_bits), len(detected_bits))
136     if n == 0:
137         return 1.0
138     return float(np.mean(np.asarray(true_bits[:n]) != np.asarray(detected_bits[:n])))
139
140
141 def evaluate_ber(mod_type, signal_data, true_bits, fs=None):
142     """
143     对给定信号执行对应解调方式并计算 BER
144
145     Parameters
146     -----
147     mod_type : str
148         'ASK' / 'FSK' / 'PSK'
149     signal_data : ndarray
150         接收信号 (含噪或滤波后)
151     true_bits : ndarray
152         发射比特序列
153
154     Returns
155     -----
156     ber : float
157         误码率
158     """
159     detected = DEMOD_FUNCS[mod_type](signal_data, fs=fs)
160     return compute_ber(true_bits, detected)

```

A.6 特征提取与调制识别 (utils/recognizer.py, 节选)

Z 域特征与时频特征提取:

```

1 def _extract_z_features(signal_data, order=6, n_samples=4000):
2     """
3     提取 Z 域特征: AR 建模主极点对
4     主极点模长反映谱线锐度 (Q 值): 恒包络调制的载波谱线尖锐 (r→1),
5     幅度键控 (ASK) 因通断切换使谱线展宽, 主极点半径明显下降
6     取信号中段多比特窗口, 保证不同符号的载频成分都能进入自相关估计
7     """
8     from utils.analyzer import signal_to_zpk
9
10    n_total = len(signal_data)

```

```

11 take = min(n_samples, n_total)
12 start = (n_total - take) // 2
13 zeros, poles, gain = signal_to_zpk(signal_data[start:start + take], order=order)
14
15 # 上半平面共轭极点对, 按模长排序
16 pairs = sorted((complex(p) for p in poles if p.imag > 1e-6),
17               key=lambda p: -abs(p))
18
19 if pairs:
20     p1 = pairs[0]
21     pole_radius = abs(p1)
22     pole_freq = abs(np.arctan2(p1.imag, p1.real)) / np.pi * config.FS / 2
23     pole_radius_2 = abs(pairs[1]) if len(pairs) > 1 else 0.0
24 else:
25     pole_radius, pole_freq, pole_radius_2 = 0.0, 0.0, 0.0
26
27 return {
28     'z_pole_radius': pole_radius,
29     'z_pole_freq': pole_freq,
30     'z_pole_radius_2': pole_radius_2,
31     'z_n_pole_pairs': len(pairs),
32 }
33
34
35 def extract_features(signal_data, fs=None):
36     """
37     提取信号的时频域特征, 用于调制方式识别
38
39     Parameters
40     -----
41     signal_data : ndarray
42         输入信号
43     fs : float
44         采样率
45
46     Returns
47     -----
48     features : dict
49         特征向量
50     """
51     fs = fs or config.FS
52     N = len(signal_data)
53
54     # ---- 时域特征 ----
55     # 包络分析
56     analytic = sig.hilbert(signal_data)
57     envelope = np.abs(analytic)
58     instantaneous_phase = np.unwrap(np.angle(analytic))
59     instantaneous_freq = np.diff(instantaneous_phase) / (2 * np.pi / fs)
60
61     # 包络统计
62     envelope_mean = np.mean(envelope)
63     envelope_var = np.var(envelope)
64     envelope_std = np.std(envelope)
65
66     # 归一化包络方差 (关键特征: ASK高, FSK/PSK低)

```

```

67     if envelope_mean > 1e-10:
68         envelope_cv = envelope_std / envelope_mean # 变异系数
69     else:
70         envelope_cv = 0
71
72     # 瞬时频率统计
73     inst_freq_mean = np.mean(instantaneous_freq)
74     inst_freq_std = np.std(instantaneous_freq)
75
76     # 过零率
77     zero_crossings = np.sum(np.diff(np.sign(signal_data)) != 0) / N
78
79     # ---- 频域特征 ----
80     # FFT
81     X = np.fft.fft(signal_data)
82     freqs = np.fft.fftfreq(N, d=1.0 / fs)
83
84     # 只取正频率
85     pos_mask = freqs > 0
86     freqs_pos = freqs[pos_mask]
87     mag_pos = np.abs(X[pos_mask])
88
89     # 限制分析范围到 [100, fs/2-100]
90     analysis_mask = (freqs_pos > 100) & (freqs_pos < fs / 2 - 100)
91     if np.sum(analysis_mask) < 10:
92         analysis_mask = np.ones_like(freqs_pos, dtype=bool)
93
94     freqs_analysis = freqs_pos[analysis_mask]
95     mag_analysis = mag_pos[analysis_mask]
96
97     # 频谱归一化
98     mag_norm = mag_analysis / (np.max(mag_analysis) + 1e-15)
99
100    # 找显著峰值 (阈值 > 0.3 * 最大值)
101    threshold = 0.3
102    peak_indices, properties = sig.find_peaks(mag_norm, height=threshold, distance=50)
103    peak_freqs = freqs_analysis[peak_indices]
104    peak_mags = mag_norm[peak_indices]
105
106    # 峰值频率数量 (关键特征: ASK=1, FSK=2, PSK=1)
107    n_peaks = len(peak_freqs)
108
109    # 主峰频率
110    if n_peaks > 0:
111        sorted_idx = np.argsort(peak_mags)[::-1]
112        main_peak_freq = peak_freqs[sorted_idx[0]]
113        if n_peaks >= 2:
114            second_peak_freq = peak_freqs[sorted_idx[1]]
115            freq_separation = abs(main_peak_freq - second_peak_freq)
116        else:
117            second_peak_freq = 0
118            freq_separation = 0
119    else:
120        main_peak_freq = 0
121        second_peak_freq = 0
122        freq_separation = 0

```

```

123
124 # 频谱集中度 (能量集中在主峰附近的比例)
125 if np.sum(mag_analysis ** 2) > 1e-15:
126     # 主峰附近  $\pm 100\text{Hz}$  范围内的能量占比
127     center = main_peak_freq
128     center_mask = (freqs_analysis > center - 100) & (freqs_analysis < center + 100)
129     spectral_concentration = np.sum(mag_analysis[center_mask] ** 2) / np.sum(mag_analysis ** 2)
130 else:
131     spectral_concentration = 0
132
133 # 频谱平坦度
134 geometric_mean = np.exp(np.mean(np.log(mag_analysis + 1e-15)))
135 arithmetic_mean = np.mean(mag_analysis)
136 spectral_flatness = geometric_mean / (arithmetic_mean + 1e-15)
137
138 # ---- 相位特征 (用于区分 PSK) ----
139 # 相位跳变检测
140 phase_diff = np.diff(instantaneous_phase)
141 phase_jumps = np.sum(np.abs(phase_diff) > np.pi / 2) / N # 归一化相位跳变率
142
143 # ---- Z 域特征 ----
144 z_feats = _extract_z_features(signal_data)
145
146 features = {
147     # 时域特征
148     'envelope_mean': envelope_mean,
149     'envelope_var': envelope_var,
150     'envelope_cv': envelope_cv,
151     'inst_freq_mean': inst_freq_mean,
152     'inst_freq_std': inst_freq_std,
153     'zero_crossing_rate': zero_crossings,
154
155     # 频域特征
156     'n_peaks': n_peaks,
157     'main_peak_freq': main_peak_freq,
158     'second_peak_freq': second_peak_freq,
159     'freq_separation': freq_separation,
160     'spectral_concentration': spectral_concentration,
161     'spectral_flatness': spectral_flatness,
162
163     # 相位特征
164     'phase_jump_rate': phase_jumps,
165
166     # Z域特征
167     **z_feats,
168
169     # 原始峰值信息
170     'peak_freqs': peak_freqs,
171     'peak_mags': peak_mags
172 }
173
174 return features

```

加权投票分类器 (支持单域/多域模式):

```

1 # ===== 调制方式识别 =====
2

```

```

3 DOMAINS = ('time', 'freq', 'z', 'multi')
4 DOMAIN_NAMES = {'time': '仅时域', 'freq': '仅频域', 'z': '仅Z域', 'multi': '多域联合'}
5
6
7 def classify_modulation(features, domain='multi'):
8     """
9     基于规则的调制方式自动分类识别
10
11     Parameters
12     -----
13     features : dict
14         由 extract_features 提取的特征
15     domain : str
16         特征域模式: 'time' 仅时域 / 'freq' 仅频域 / 'z' 仅Z域 /
17         'multi' 多域联合。用于单一域 vs 多域联合的对比消融实验
18
19     Returns
20     -----
21     result : dict
22         分类结果, 包含类型和置信度
23     """
24     n_peaks = features['n_peaks']
25     envelope_cv = features['envelope_cv']
26     freq_separation = features['freq_separation']
27     phase_jump_rate = features['phase_jump_rate']
28     spectral_concentration = features['spectral_concentration']
29
30     use_freq = domain in ('freq', 'multi')
31     use_time = domain in ('time', 'multi')
32     use_z = domain in ('z', 'multi')
33
34     scores = {'ASK': 0.0, 'FSK': 0.0, 'PSK': 0.0}
35     reasons = {'ASK': [], 'FSK': [], 'PSK': []}
36
37     # ---- 规则1 [频域]: 频谱峰值数量 ----
38     if use_freq:
39         if n_peaks >= 2 and freq_separation > 200:
40             scores['FSK'] += 3.0
41             reasons['FSK'].append(f'频谱双峰(间隔{freq_separation:.0f}Hz)')
42         elif n_peaks == 1:
43             scores['ASK'] += 1.0
44             scores['PSK'] += 1.0
45             reasons['ASK'].append('频谱单峰')
46             reasons['PSK'].append('频谱单峰')
47
48     # ---- 规则2 [时域]: 包络变异系数 ----
49     if use_time:
50         if envelope_cv > 0.5:
51             scores['ASK'] += 3.0
52             reasons['ASK'].append(f'包络变化大(CV={envelope_cv:.3f})')
53         elif envelope_cv < 0.3:
54             scores['FSK'] += 1.0
55             scores['PSK'] += 1.5
56             reasons['FSK'].append(f'包络稳定(CV={envelope_cv:.3f})')
57             reasons['PSK'].append(f'包络稳定(CV={envelope_cv:.3f})')
58

```

```

59 # ---- 规则3 [时域]: 相位跳变率 ----
60 if use_time:
61     if phase_jump_rate > 0.005:
62         scores['PSK'] += 2.5
63         reasons['PSK'].append(f'相位跳变频繁(rate={phase_jump_rate:.4f})')
64     else:
65         scores['ASK'] += 0.5
66         scores['FSK'] += 0.5
67         reasons['ASK'].append(f'相位跳变少')
68         reasons['FSK'].append(f'相位跳变少')
69
70 # ---- 规则4 [时域]: 瞬时频率标准差 ----
71 if use_time:
72     inst_freq_std = features['inst_freq_std']
73     if inst_freq_std > 300:
74         scores['FSK'] += 2.0
75         reasons['FSK'].append(f'瞬时频率变化大(std={inst_freq_std:.0f})')
76
77 # ---- 规则5 [频域]: 频谱集中度 ----
78 if use_freq:
79     if spectral_concentration > 0.6:
80         scores['ASK'] += 1.0
81         scores['PSK'] += 1.0
82         reasons['ASK'].append(f'频谱集中(conc={spectral_concentration:.3f})')
83
84 # ---- 规则6 [Z域]: 主极点半径 (谱线锐度) ----
85 # Z 域可区分"幅度键控(谱线展宽)"与"恒包络(尖锐谐振)"两大类,
86 # 作为轻量佐证参与融合; 恒包络内部 FSK/PSK 在 Z 域不可分
87 if use_z:
88     r1 = features.get('z_pole_radius', 0.0)
89     if r1 >= 0.96:
90         scores['FSK'] += 1.5
91         scores['PSK'] += 1.5
92         reasons['FSK'].append(f'Z域尖锐谐振极点(r={r1:.3f},恒包络)')
93         reasons['PSK'].append(f'Z域尖锐谐振极点(r={r1:.3f},恒包络)')
94     elif r1 > 0:
95         scores['ASK'] += 1.5
96         reasons['ASK'].append(f'Z域谱线展宽(r={r1:.3f},幅度键控)')
97
98 # ---- 规则7 [多域协同]: 单峰 + 恒包络 + 窄带瞬时频率 + PSK ----
99 if domain == 'multi':
100     if (n_peaks == 1 and envelope_cv < 0.3
101         and features['inst_freq_std'] < 300 and spectral_concentration > 0.6):
102         scores['PSK'] += 1.5
103         reasons['PSK'].append('单峰恒包络窄带特征')
104
105 # 确定最终分类
106 predicted = max(scores, key=scores.get)
107 total = sum(scores.values())
108 confidence = scores[predicted] / total if total > 0 else 0
109
110 result = {
111     'predicted': predicted,
112     'confidence': confidence,
113     'domain': domain,
114     'scores': scores,

```

```

115     'reasons': reasons[predicted],
116     'all_reasons': reasons,
117     'features': features
118 }
119
120 return result
121
122
123 def classify_signal(signal_data, fs=None, domain='multi'):
124     """
125     端到端调制方式识别：特征提取 + 分类
126     """
127     features = extract_features(signal_data, fs)
128     result = classify_modulation(features, domain=domain)
129     return result

```

单域 vs 多域消融实验：

```

1 def run_domain_ablation(snr_range=None, n_trials=None):
2     """
3     单一域 vs 多域联合识别消融实验
4
5     同一特征提取结果分别在 仅时域/仅频域/仅Z域/多域联合 四种模式下
6     分类，统计各 SNR 下的平均识别准确率，量化多域融合的增益
7
8     Returns
9     -----
10    ablation : dict
11        {'snr_range', 'n_trials', 'accuracy': {domain: [per-snr acc]}}
12    """
13    from utils import generator, filter_design as fd
14
15    snr_range = snr_range or config.SNR_RANGE
16    n_trials = n_trials or config.SWEEP_TRIALS
17    mod_types = ['ASK', 'FSK', 'PSK']
18
19    accuracy = {d: [] for d in DOMAINS}
20
21    for snr_db in snr_range:
22        correct = {d: 0 for d in DOMAINS}
23        total = 0
24
25        for trial in range(n_trials):
26            signals = generator.generate_all_signals(
27                n_bits=config.N_BITS, snr_db=snr_db,
28                seed=config.RANDOM_SEED + trial * 100 + int(snr_db)
29            )
30
31            for mod_type, data in signals.items():
32                # 统一使用 FIR 滤波后的信号进行识别
33                b, a = fd.design_fir(config.FIR_DEFAULT_ORDER,
34                                    config.BANDPASS_LOW_FSK,
35                                    config.BANDPASS_HIGH_FSK,
36                                    config.FS, config.FIR_DEFAULT_WINDOW)
37                filtered = fd.apply_filter(b, a, data['noisy'])
38
39                features = extract_features(filtered, config.FS)

```

```

40         for d in DOMAINS:
41             rec = classify_modulation(features, domain=d)
42             correct[d] += int(rec['predicted'] == mod_type)
43             total += 1
44
45         for d in DOMAINS:
46             accuracy[d].append(correct[d] / total)
47
48         line = ' | '.join(f'{DOMAIN_NAMES[d]}={accuracy[d][-1]:.0%}' for d in DOMAINS)
49         print(f" [SNR={snr_db}>3]dB] {line}")
50
51     return {'snr_range': list(snr_range), 'n_trials': n_trials,
52           'accuracy': accuracy}

```

A.7 主流程 (main.py)

```

1  def separator(title):
2      """打印分隔线"""
3      print(f"\n{'='*70}")
4      print(f" {title}")
5      print(f"{'='*70}")
6
7
8  def main():
9      """主函数：执行完整的 DSP 处理流程"""
10
11      start_time = time.time()
12      print("=" * 70)
13      print(" 多变换域联合的信号调制特征识别与滤波优化系统")
14      print(" Multi-Transform Domain Joint Signal Modulation Recognition System")
15      print("=" * 70)
16
17      # =====
18      # Step 1: 生成 ASK/FSK/PSK 信号
19      # =====
20      separator("Step 1: 调制信号建模与生成")
21
22      print(f" 采样率: {config.FS} Hz")
23      print(f" 比特数: {config.N_BITS}")
24      print(f" 每比特采样点数: {config.SAMPLES_PER_BIT}")
25      print(f" 信噪比: {config.SNR_DB} dB")
26
27      signals = generator.generate_all_signals(
28          n_bits=config.N_BITS, snr_db=config.SNR_DB, seed=config.RANDOM_SEED
29      )
30
31      for mod_type, data in signals.items():
32          print(f" [{mod_type}] 信号长度: {len(data['clean'])} 样本, "
33                f"时长: {len(data['clean'])/config.FS*1000:.1f} ms")
34
35      print(" 信号生成完成")
36
37      # =====
38      # Step 2: 时域分析
39      # =====

```



```

40 separator("Step 2: 时域波形分析与卷积平滑")
41
42 # 2.1 时域波形对比
43 analyzer.plot_time_domain_signals(
44     signals,
45     save_path=os.path.join(config.PLOTS_DIR, '01_time_domain_signals.png')
46 )
47
48 # 2.2 卷积平滑效果对比
49 analyzer.plot_convolution_smooth(
50     signals,
51     kernel_sizes=config.SMOOTH_KERNELS,
52     save_path=os.path.join(config.PLOTS_DIR, '02_convolution_smooth.png')
53 )
54
55 # 时域特征提取
56 print("\n 时域特征统计: ")
57 for mod_type, data in signals.items():
58     features = analyzer.time_domain_features(data['clean'])
59     print(f" [{mod_type}] 均值={features['mean']:.4f}, "
60           f"方差={features['variance']:.4f}, "
61           f"RMS={features['rms']:.4f}, "
62           f"包络方差={features['envelope_var']:.4f}, "
63           f"过零数={features['zero_crossings']}")
64
65 print(" 时域分析完成")
66
67 # =====
68 # Step 3: 多域联合分析
69 # =====
70 separator("Step 3: 多变换域联合特征分析")
71
72 # 3.1 DFT 离散频谱
73 analyzer.plot_dft_spectrum(
74     signals,
75     save_path=os.path.join(config.PLOTS_DIR, '03_dft_spectrum.png')
76 )
77 print(" DFT 离散频谱分析完成")
78
79 # 3.2 DTFT 连续频谱
80 analyzer.plot_dtft_spectrum(
81     signals,
82     save_path=os.path.join(config.PLOTS_DIR, '04_dtft_spectrum.png')
83 )
84 print(" DTFT 连续频谱分析完成")
85
86 # 3.3 Z 变换零极点
87 analyzer.plot_zero_pole_map(
88     signals,
89     save_path=os.path.join(config.PLOTS_DIR, '05_zero_pole_map.png')
90 )
91 print(" Z 变换零极点分析完成")
92
93 # 3.4 多域联合综合对比
94 analyzer.plot_multi_domain_comparison(
95     signals,

```

```

96     save_path=os.path.join(config.PLOTS_DIR, '13_multi_domain_comparison.png')
97 )
98 print(" 多域联合对比图生成完成")
99
100 # 3.5 三种变换应用差异对比 (DFT DTFT 采样关系 + Z域谐振特征)
101 analyzer.plot_dft_dtft_comparison(
102     signals,
103     save_path=os.path.join(config.PLOTS_DIR, '15_transform_comparison.png')
104 )
105 print("\n 三种变换分工: ")
106 print(" • DTFT(连续谱) → 观察谱形与带宽, 零填充FFT逼近")
107 print(" • DFT(离散谱线) → 频率分辨率 FS/N, 支撑峰值检测与功率分布统计")
108 print(" • Z变换(零极点) → 极点角位置对应谐振频率, 揭示系统级特征")
109
110 # =====
111 # Step 4: 滤波器设计与特性分析
112 # =====
113 separator("Step 4: FIR/IIR 滤波器设计与频率响应特性")
114
115 # 定义各调制方式的带通参数
116 bandpass_params = {
117     'ASK': (config.BANDPASS_LOW_ASK, config.BANDPASS_HIGH_ASK),
118     'FSK': (config.BANDPASS_LOW_FSK, config.BANDPASS_HIGH_FSK),
119     'PSK': (config.BANDPASS_LOW_PSK, config.BANDPASS_HIGH_PSK),
120 }
121
122 # 4.1 设计并绘制 FIR 滤波器
123 print("\n FIR 滤波器设计 (窗函数法):")
124 fir_filters = {}
125 for mod_type in signals:
126     low, high = bandpass_params[mod_type]
127     b, a = filter_design.design_fir(
128         config.FIR_DEFAULT_ORDER, low, high, config.FS, config.FIR_DEFAULT_WINDOW
129     )
130     fir_filters[mod_type] = (b, a)
131     print(f" [{mod_type}] FIR order={config.FIR_DEFAULT_ORDER}, "
132           f"window={config.FIR_DEFAULT_WINDOW}, "
133           f"cutoff=[{low}, {high}] Hz, 系数数量={len(b)}")
134
135 # 绘制 FIR 频率响应 (以 FSK 为例展示完整特性)
136 b_fsk, a_fsk = fir_filters['FSK']
137 filter_design.plot_filter_response(
138     b_fsk, a_fsk, config.FS,
139     title=f'(FIR, order={config.FIR_DEFAULT_ORDER}, {config.FIR_DEFAULT_WINDOW})',
140     save_path=os.path.join(config.PLOTS_DIR, '06_fir_response.png')
141 )
142 print("\n [系统函数推导] FIR 带通 (FSK):")
143 print(analyzer.format_system_function(b_fsk, a_fsk))
144
145 # 4.2 设计并绘制 IIR 滤波器
146 print("\n IIR 滤波器设计:")
147 iir_filters = {}
148 for mod_type in signals:
149     low, high = bandpass_params[mod_type]
150     b, a = filter_design.design_iir(
151         config.IIR_DEFAULT_ORDER, low, high, config.FS, config.IIR_DEFAULT_TYPE

```

```

152     )
153     iir_filters[mod_type] = (b, a)
154     print(f" [{mod_type}] IIR order={config.IIR_DEFAULT_ORDER}, "
155           f"type={config.IIR_DEFAULT_TYPE}, "
156           f"cutoff=[{low}, {high}] Hz")
157
158     # 绘制 IIR 频率响应
159     b_fsk_iir, a_fsk_iir = iir_filters['FSK']
160     filter_design.plot_filter_response(
161         b_fsk_iir, a_fsk_iir, config.FS,
162         title=f'(IIR, order={config.IIR_DEFAULT_ORDER}, {config.IIR_DEFAULT_TYPE})',
163         save_path=os.path.join(config.PLOTS_DIR, '07_iir_response.png')
164     )
165     print("\n [系统函数推导] IIR 带通 (FSK):")
166     print(analyzer.format_system_function(b_fsk_iir, a_fsk_iir))
167
168     # 绘制滤波器零点极点图
169     filter_design.plot_filter_zero_pole(
170         b_fsk_iir, a_fsk_iir,
171         title=f'(IIR Butterworth, order={config.IIR_DEFAULT_ORDER})',
172         save_path=os.path.join(config.PLOTS_DIR, '05b_filter_zero_pole.png')
173     )
174
175     print(" 滤波器设计完成")
176
177     # 4.3 滤波并绘制对比
178     print("\n 执行滤波操作...")
179     filtered_results = {} # {mod_type: {method: (signal, time)}}
180
181     for mod_type in signals:
182         data = signals[mod_type]
183         filtered_results[mod_type] = {}
184
185         # FIR 滤波
186         b, a = fir_filters[mod_type]
187         t0 = time.time()
188         fir_filtered = filter_design.apply_filter(b, a, data['noisy'])
189         t_fir = time.time() - t0
190         filtered_results[mod_type]['FIR最优'] = (fir_filtered, t_fir)
191
192         # IIR 滤波
193         b, a = iir_filters[mod_type]
194         t0 = time.time()
195         iir_filtered = filter_design.apply_filter(b, a, data['noisy'])
196         t_iir = time.time() - t0
197         filtered_results[mod_type]['IIR最优'] = (iir_filtered, t_iir)
198
199     # 绘制滤波前后对比 (每种调制方式各一张图)
200     for mod_type in signals:
201         data = signals[mod_type]
202         fir_f = filtered_results[mod_type]['FIR最优'][0]
203         iir_f = filtered_results[mod_type]['IIR最优'][0]
204
205         filter_design.plot_filter_comparison(
206             data['clean'], data['noisy'], fir_f, iir_f, data['t'],
207             mod_type=mod_type,

```

```

208     save_path=os.path.join(config.PLOTS_DIR, f'08_{mod_type}_filter_comparison.png')
209 )
210
211 print(" 滤波与对比图生成完成")
212
213 # =====
214 # Step 5: 滤波器迭代优化
215 # =====
216 separator("Step 5: 滤波器参数迭代优化")
217
218 optimization_results = {}
219 best_filters = {}
220
221 for mod_type in signals:
222     data = signals[mod_type]
223     low, high = bandpass_params[mod_type]
224
225     print(f"\n [{mod_type}] 开始参数扫描...")
226     results, best = filter_design.iterative_optimize(
227         data['clean'], data['noisy'], config.FS,
228         cutoff_low=low, cutoff_high=high
229     )
230
231     optimization_results[mod_type] = results
232     best_filters[mod_type] = best
233
234     if best:
235         scale_info = f", 带宽×{best['cutoff_scale']:g}" if best.get('cutoff_scale', 1) != 1 else ""
236         print(f" [{mod_type}] 最优: {best['type']} order={best['order']} "
237               f"{best['window']}{scale_info}, "
238               f"SNR改善={best['snr_improvement']:.2f}dB, "
239               f"保真度={best['correlation']:.4f}")
240
241         # 使用最优滤波器的结果更新
242         filtered_results[mod_type][f'{best["type"]}迭代最优'] = (
243             best['filtered'], best['time']
244         )
245
246         # 最优滤波器幅频特性 + Z域零极点分析
247         filter_design.plot_best_filter_analysis(
248             best['b'], best['a'], best['cutoff'], mod_type=mod_type,
249             best_info=best,
250             save_path=os.path.join(config.PLOTS_DIR, f'09b_{mod_type}_best_filter.png')
251         )
252
253         # 绘制优化结果
254         filter_design.plot_optimization_results(
255             results, mod_type=mod_type,
256             save_path=os.path.join(config.PLOTS_DIR, f'09_{mod_type}_optimization.png')
257         )
258
259 print(" 迭代优化完成")
260
261 # =====
262 # Step 6: NLMS 归一化自适应滤波 (创新拓展)
263 # =====

```

```

264 separator("Step 6: NLMS 归一化自适应滤波 (创新拓展)")
265
266 all_lms_results = {}
267
268 for mod_type in signals:
269     data = signals[mod_type]
270
271     print(f"\n [{mod_type}] NLMS 自适应滤波分析...")
272
273     # 评估不同归一化步长 (步长与输入功率无关, 收敛快于标准 LMS)
274     lms_results = filter_design.evaluate_lms_mu(
275         data['clean'], data['noisy'], data['noise'],
276         mu_range=config.NLMS_MU_RANGE,
277         order=config.NLMS_ORDER,
278         algorithm='nlms'
279     )
280     all_lms_results[mod_type] = lms_results
281
282     # 绘制 NLMS 收敛分析
283     filter_design.plot_lms_convergence(
284         lms_results, mod_type=mod_type,
285         save_path=os.path.join(config.PLOTS_DIR, f'10_{mod_type}_lms_convergence.png')
286     )
287
288     # 找最优步长
289     best_lms = max(lms_results, key=lambda x: x['snr_out'])
290     print(f" [{mod_type}] 最优步长: ={best_lms['mu']:.3f}, "
291           f"SNR={best_lms['snr_out']:.2f}dB "
292           f"(改善{best_lms['snr_improvement']:.2f}dB), "
293           f"相关系数={best_lms['correlation']:.4f}, "
294           f"收敛样本数={best_lms['convergence_samples']}")
295
296     # 使用最优步长进行去噪 (计时并复用结果, 避免重复计算)
297     t0 = time.time()
298     denoised, mse_hist, weights, wh = filter_design.lms_denoise(
299         data['noisy'], data['noise'],
300         mu=best_lms['mu'], order=config.NLMS_ORDER, algorithm='nlms'
301     )
302     t_lms = time.time() - t0
303
304     # 绘制 NLMS 波形对比
305     filter_design.plot_lms_detailed_waveform(
306         data['clean'], data['noisy'], denoised, data['t'],
307         mod_type=mod_type, mu=best_lms['mu'],
308         save_path=os.path.join(config.PLOTS_DIR, f'10b_{mod_type}_lms_waveform.png')
309     )
310
311     filtered_results[mod_type]['NLMS自适应'] = (denoised, t_lms)
312
313 print(" NLMS 自适应滤波分析完成")
314
315 # =====
316 # Step 7: 信号特征识别
317 # =====
318 separator("Step 7: ASK/FSK/PSK 自动识别")
319

```

```

320     recognition_results = {}
321
322     for mod_type in signals:
323         data = signals[mod_type]
324
325         # 使用最优滤波器的输出进行识别
326         if best_filters[mod_type]:
327             signal_for_recognition = best_filters[mod_type]['filtered']
328         else:
329             signal_for_recognition = data['noisy']
330
331         result = recognizer.classify_signal(signal_for_recognition, config.FS)
332         recognition_results[mod_type] = result
333
334         is_correct = result['predicted'] == mod_type
335         status = " 正确" if is_correct else " 错误"
336
337         print(f" [{mod_type}] → 识别为: {result['predicted']} "
338               f"(置信度: {result['confidence']:.1%}) {status}")
339         print(f" 依据: {'', '.join(result['reasons'])}")
340
341     # 绘制识别结果
342     recognizer.plot_recognition_results(
343         recognition_results,
344         save_path=os.path.join(config.PLOTS_DIR, '11_recognition_result.png')
345     )
346
347     print(" 信号识别完成")
348
349     # =====
350     # Step 8: 性能评估 (含误码率 BER)
351     # =====
352     separator("Step 8: 综合性能评估 (SNR / 保真度 / 误码率 BER) ")
353
354     # 准备数据
355     clean_signals = {mod: signals[mod]['clean'] for mod in signals}
356     noisy_signals = {mod: signals[mod]['noisy'] for mod in signals}
357
358     # 未滤波基线 (对照)
359     for mod_type in signals:
360         filtered_results[mod_type]['未滤波'] = (signals[mod_type]['noisy'], 0.0)
361
362     # 评估所有方法 (含解调级 BER)
363     all_metrics = recognizer.evaluate_all_methods(
364         clean_signals, noisy_signals, filtered_results,
365         {mod: signals[mod]['t'] for mod in signals},
366         true_bits={mod: signals[mod]['bits'] for mod in signals}
367     )
368
369     # 运算复杂度: 每输出样本的乘加次数 (零相位滤波 filtfilt 双向 ×2)
370     for mod_type in all_metrics:
371         for method in all_metrics[mod_type]:
372             if method == '未滤波':
373                 ops = 0.0
374             elif method == 'NLMS自适应':
375                 ops = 4.0 * config.NLMS_ORDER

```

```

376         elif 'FIR' in method:
377             if method == 'FIR迭代最优':
378                 b_len = len(best_filters[mod_type]['b']) if best_filters[mod_type] else 0
379                 ops = 2.0 * b_len
380             else:
381                 ops = 2.0 * len(fir_filters[mod_type][0])
382             else:
383                 b_iir, a_iir = iir_filters[mod_type]
384                 ops = 2.0 * (len(b_iir) + len(a_iir) - 1)
385                 all_metrics[mod_type][method]['ops_per_sample'] = ops
386
387     for mod_type in all_metrics:
388         print(f"\n [{mod_type}]")
389         for method, m in all_metrics[mod_type].items():
390             ber_str = f", BER={m['ber']:.4f}" if m['ber'] is not None else ""
391             print(f"    {method:<12} ΔSNR={m['snr_improvement']:+6.2f}dB, "
392                   f"    保真度={m['fidelity']:.4f}{ber_str}")
393
394     # 绘制性能汇总
395     recognizer.plot_performance_summary(
396         all_metrics,
397         save_path=os.path.join(config.PLOTS_DIR, '12_performance_summary.png')
398     )
399
400     print("\n 性能评估完成")
401
402     # =====
403     # Step 9: SNR 扫描实验
404     # =====
405     separator("Step 9: SNR 扫描实验（滤波性能 + 调制识别准确率）")
406
407     print(f" 扫描范围: {config.SNR_RANGE} dB, "
408           f" 每点 {config.SWEEP_TRIALS} 次蒙特卡洛试验\n")
409
410     sweep_results = recognizer.run_snr_sweep(
411         snr_range=config.SNR_RANGE, n_trials=config.SWEEP_TRIALS
412     )
413
414     recognizer.plot_snr_sweep_results(
415         sweep_results,
416         save_path=os.path.join(config.PLOTS_DIR, '14_snr_sweep.png')
417     )
418
419     print("\n  SNR 扫描实验完成")
420
421     # =====
422     # Step 10: 单一域 vs 多域联合识别消融实验
423     # =====
424     separator("Step 10: 单一域 vs 多域联合识别消融实验")
425
426     print(f" 对比模式: 仅时域 / 仅频域 / 仅Z域 / 多域联合, "
427           f" 每点 {config.SWEEP_TRIALS}×3 次试验\n")
428
429     ablation_results = recognizer.run_domain_ablation(
430         snr_range=config.SNR_RANGE, n_trials=config.SWEEP_TRIALS
431     )

```

```

432
433 recognizer.plot_domain_ablation(
434     ablation_results,
435     save_path=os.path.join(config.PLOTS_DIR, '16_domain_ablation.png')
436 )
437
438 print("\n 消融实验完成")
439
440 # =====
441 # Step 11: 报告生成
442 # =====
443 separator("Step 11: 生成评估报告")
444
445 report = recognizer.generate_report(
446     recognition_results, all_metrics, all_lms_results,
447     sweep_results=sweep_results, ablation_results=ablation_results,
448     log_file=config.LOG_FILE
449 )
450
451 # =====
452 # 完成
453 # =====
454 total_time = time.time() - start_time
455
456 separator("完成!")
457 print(f" 总耗时: {total_time:.2f} 秒")
458 print(f" 图表输出: {config.PLOTS_DIR}")
459 print(f" 评估报告: {config.LOG_FILE}")
460
461 # 列出所有生成的图表
462 plots = sorted(os.listdir(config.PLOTS_DIR))
463 print(f"\n 生成图表清单 ({len(plots)} 张):")
464 for p in plots:
465     print(f"    • {p}")
466
467 print(f"\n{'='*70}")
468 print(" 程序运行结束")
469 print(f"{'='*70}")
470
471
472 if __name__ == '__main__':
473     main()

```